THE COMPLETE TUTORIAL

Python Programming

From scratch to the very advanced level: a complete, example driven tutorial

17 MODULES | OVER 200 SLIDES | EVERY EXAMPLE EXECUTED

Beginner → Intermediate → Advanced

PYTHON

```python
def learn(topic):
    return f"I can use {topic}!"

for step in ["loops", "classes", "asyncio"]:
    print(learn(step))
```

OUTPUT

```
I can use loops!
I can use classes!
I can use asyncio!
```

How to read every slide

PYTHON

```
price = 4.50
cups = 3
print("Total:", price * cups)
```

1

OUTPUT

```
Total: 13.5
```

THREE KINDS OF PANEL

Dark panels hold code to type. Pale panels show what it prints. A yellow circle tags a line that a callout explains.

KEY IDEA

The one sentence you must remember from the slide.

TIP

A habit or shortcut that experienced developers use.

WATCH OUT

A classic mistake, shown before it can bite you.

UNDER THE HOOD

What the interpreter really does behind the scenes.

IN PLAIN ENGLISH

The code retold as an ordinary sentence.

TRY IT YOURSELF

A short exercise. Type it, run it, then change it.

VOCABULARY

Term: a new word, defined the moment it appears.

LINE BY LINE

A guided walk through the code, one line at a time.

The road ahead: seventeen modules

BEGINNER

- 01 Getting Started
- 02 Variables and Data Types
- 03 Operators and Expressions
- 04 Control Flow
- 05 Data Structures

INTERMEDIATE

- 06 Functions
- 07 Modules and Packages
- 08 Files and Data Formats
- 09 Errors and Exceptions
- 10 Object Oriented Programming

ADVANCED

- 11 Iterators and Generators
- 12 Decorators
- 13 Type Hints
- 14 Python Internals
- 15 Concurrency
- 16 Testing and Speed
- 17 Capstone Project

What Python is and why it matters

Python is a general purpose, high level programming language created by Guido van Rossum and first released in 1991. This tutorial targets Python 3.12 and later.

1 Readable by design

Python code looks close to plain English. Indentation shows structure, so programs stay tidy and easy to review.

2 Batteries included

The standard library ships with tools for files, text, math, dates, networking, testing and much more.

3 One language, many jobs

Web services, automation, data analysis, machine learning, scientific computing and scripting all use Python.

4 A giant ecosystem

The Python Package Index hosts hundreds of thousands of free libraries that you can install with one command.

VOCABULARY

Interpreter: the program that reads your Python code and carries it out, statement by statement.

VOCABULARY

High level language: one that hides hardware details such as memory addresses so that you can focus on the problem.

How to practise with this tutorial

Programming is a practical skill. Reading alone will not make it stick, so treat every slide as a small experiment.

1 Read

Study the code panel slowly. Every symbol is there for a reason.

2 Predict

Cover the output panel and say aloud what you expect to see.

3 Type

Type the example yourself. Copying by hand builds muscle memory.

4 Break

Change one thing and run again. Read the error message with care.

5 Extend

Finish the exercise in the green box before you move to the next slide.

KEY IDEA

Understanding grows fastest when a prediction turns out to be wrong. Wrong guesses are the most valuable part of practice.

TIP

Keep one file named **scratch.py** open at all times. Use it to test every idea the moment you have it.

01

MODULE 01 BEGINNER

Getting Started

Install Python, run your first program and learn how the interpreter reads your code.

IN THIS MODULE

1. Installing and launching Python
2. The interactive shell and script files
3. The structure of a statement
4. Comments and indentation
5. How Python executes a program
6. Reading your first error message

Installing Python and checking it works

TERMINAL

```
# 1. Download the installer from python.org and run it
#    (on Windows, tick "Add python.exe to PATH")

# 2. Open a terminal and ask for the version
python --version

# 3. On macOS and Linux the command is often python3
python3 --version

# 4. Start the interactive shell
python
```

OUTPUT

```
Python 3.12.3
```

WHAT YOU NEED

- Python 3.12 or newer.
- A terminal: PowerShell, Terminal or any shell.
- A code editor such as VS Code, PyCharm or IDLE, which ships with Python.

VOCABULARY

Terminal: a text window where you type commands for the operating system instead of clicking.

WATCH OUT

If the terminal answers that the command was not found, Python is not on your PATH. Run the installer again and tick the PATH option.

Two ways to run Python code

INTERACTIVE SHELL

```
>>> 2 + 3
5
>>> name = "Ada"
>>> name.upper()
'ADA'
>>> exit()
```

SCRIPT FILE: greet.py

```
name = "Ada"
print(name.upper())
print(2 + 3)
```

RUN WITH: python greet.py

```
ADA
5
```

THE SHELL

Type one line, press Enter and see the result at once. Nothing is saved. Perfect for quick experiments.

THE SCRIPT

Save many lines in a file that ends in **.py** and run them together. This is how real programs are built.

VOCABULARY

REPL: the shell runs a loop that will Read your line, Evaluate it, Print the result and Loop again.

Your first program

PYTHON

```python
# hello.py: my first Python program
print("Hello, World!")

name = "Ada"
print("Welcome,", name)
print("2 + 3 =", 2 + 3)
```

1 2 3 4

OUTPUT

```
Hello, World!
Welcome, Ada
2 + 3 = 5
```

LINE BY LINE

1. **print()** is a built in function that writes text to the screen.
2. The equals sign stores the text **Ada** under the name **name**.
3. Several values separated by commas are printed with a space between them.
4. Python evaluates **2 + 3** first and then prints the result.

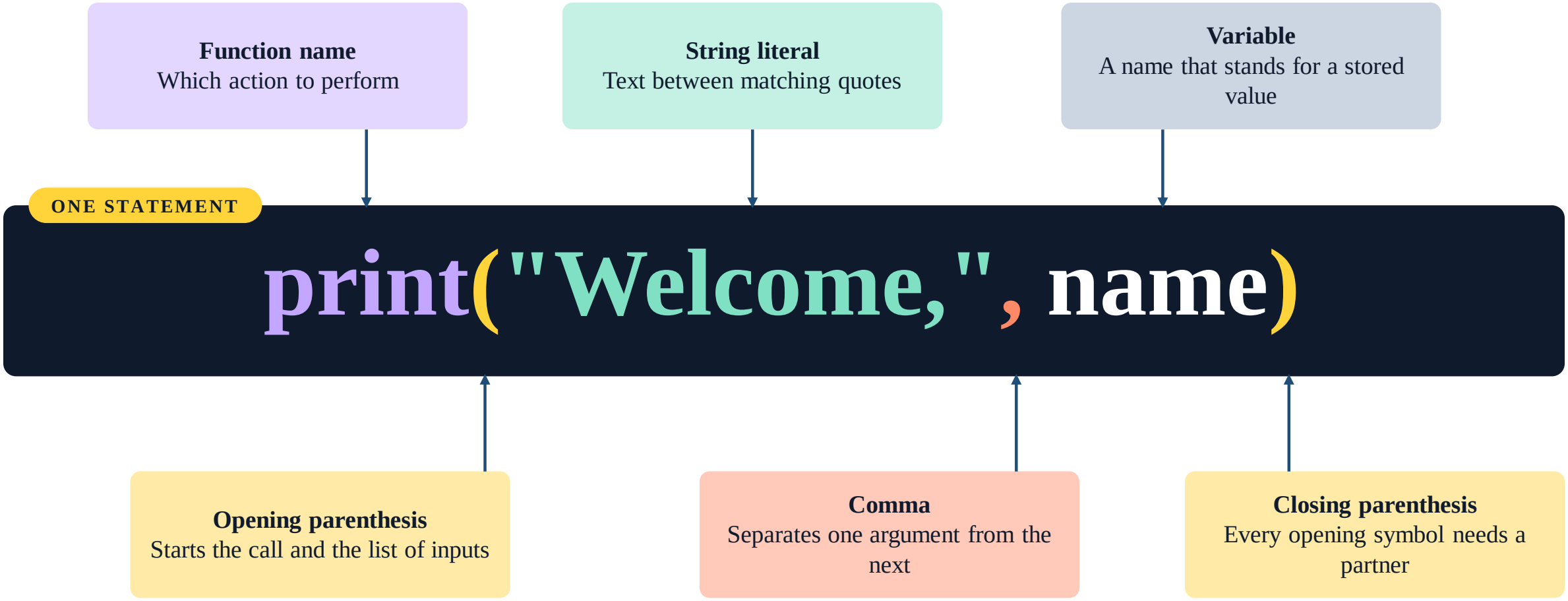
IN PLAIN ENGLISH

Greet the world, remember the name Ada, welcome her and show a small sum.

TRY IT YOURSELF

Change the name to your own and add a fourth line that prints your favourite number doubled.

The structure of a statement



Comments and docstrings

PYTHON

```
# A comment starts with a hash and runs to the end of the line.
radius = 5   # comments can follow code as well

def circle_area(r):
    """Return the area of a circle with radius r."""
    return 3.14159 * r * r

print(circle_area(radius))
print(circle_area.__doc__)
```

OUTPUT

```
78.53975
Return the area of a circle with radius r.
```

COMMENTS

Python ignores comments completely. They exist for human readers, including you in six months.

DOCSTRINGS

A triple quoted string placed first inside a function becomes its documentation, stored in **__doc__** and shown by **help()**.

TIP

Good comments explain **why** the code does something. The code itself already shows what it does.

WATCH OUT

An outdated comment is worse than no comment. Update the comment whenever you change the code.

Indentation is part of the syntax

CORRECT

```
temperature = 31
if temperature > 30:
    print("It is hot")
    print("Drink some water")
print("Done")
```

BROKEN

```
temperature = 31
if temperature > 30:
print("It is hot")
print("Done")
```

OUTPUT

```
It is hot
Drink some water
Done
```

ERROR

```
IndentationError: expected an indented block after 'if'
statement on line 2
```

KEY IDEA

A colon opens a block. Every line that belongs to the block is indented by the same amount, and the block ends when the indentation returns.

TIP

Use four spaces per level. Set your editor to insert spaces when you press the Tab key.

WATCH OUT

Never mix tabs and spaces in one file. Python refuses to guess how wide a tab is.

How Python executes a program

Python is called an interpreted language, yet a quick compile step always happens first. You never need to run it by hand.

1 Source

You save plain text in a file such as **hello.py**.

2 Parse

Python checks the grammar. A mistake here is a **SyntaxError** and nothing runs.

3 Compile

Valid code is translated into compact instructions called bytecode.

4 Execute

The Python virtual machine runs the bytecode, one instruction at a time, top to bottom.

5 Result

Text appears, files change and values are produced.

UNDER THE HOOD

Bytecode for imported modules is cached in a folder named **__pycache__**, so the second start of a program is faster.

VOCABULARY

CPython: the standard interpreter from python.org, written in C. It is the one this tutorial describes.

Reading your first error message

PYTHON

```
print("Start")
total = 10 + "5"
print("End")
```

!

OUTPUT

```
Start
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    total = 10 + "5"
TypeError: unsupported operand type(s) for +: 'int' and
'str'
```

HOW TO READ IT

1. Start at the **last line**. It names the error type and gives the reason.
2. Move up one step to find the file, the line number and the offending code.
3. Lines printed before the traceback ran successfully.

KEY IDEA

Python stopped at line 2, so line 3 never ran. An unhandled error ends the program at once.

TIP

Errors are information, not failure. Read every word of the message before you change the code.

Three families of mistakes

BEFORE THE PROGRAM RUNS

Syntax errors

The grammar is wrong, so Python cannot even start. Typical causes are a missing colon, quote or parenthesis.

```
print("Hello"
```

WHILE THE PROGRAM RUNS

Runtime errors

The grammar is fine but an operation is impossible. Python raises an exception and prints a traceback.

```
print(10 / 0)
```

NO MESSAGE AT ALL

Logic errors

The program runs and gives a wrong answer. Only testing and careful thought reveal these.

```
average = 10 + 20 / 2
```

TIP

Fix the **first** error that Python reports. Later messages are often just consequences of the first one.

TRY IT YOURSELF

Type each snippet into the shell. Which exception names appear for the first two? Why is the third one wrong?

Quiz: predict the output

PREDICT THE OUTPUT

```
print("A")
# print("B")
print("C")   # print("D")
```

ACTUAL OUTPUT

```
A
C
```

QUESTION

What does this program print?

OPTIONS

A. **A B C D** on four lines

B. **A C** on two lines

C. **A C D** on three lines

D. Nothing, because of an error

ANSWER

B. Everything after a hash is a comment, so only the first and third calls run.

Module 1 recap

1. Python code can run line by line in the interactive shell or all at once from a script file.
2. **print()** writes values to the screen. Commas separate the values to print.
3. Comments start with a hash. A docstring in triple quotes documents a function.
4. Indentation is syntax. A colon opens a block and four spaces mark its body.
5. Source code is parsed, compiled to bytecode and then executed top to bottom.
6. A traceback is read from the bottom up: error type first, then the location.

Coming next: Module 2 introduces variables, numbers, strings and the other core data types.

02

MODULE 02 BEGINNER

Variables and Data Types

Give names to values and meet the building blocks of every program: numbers, text, truth values and None.

IN THIS MODULE

1. Variables as name tags
2. Integers and floats
3. Strings, indexing and slicing
4. String methods and f strings
5. Booleans and None
6. Type conversion and input

Variables are name tags, not boxes

PYTHON

```
x = 42
y = x
x = 99
print(x, y)
```

1
2

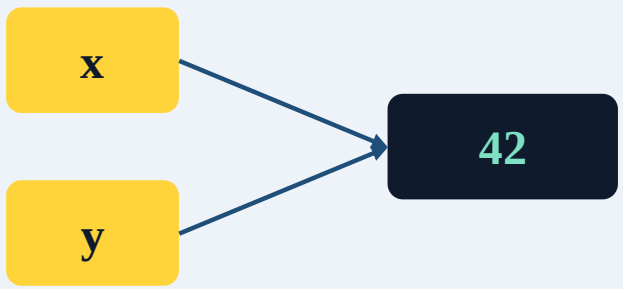
OUTPUT

```
99 42
```

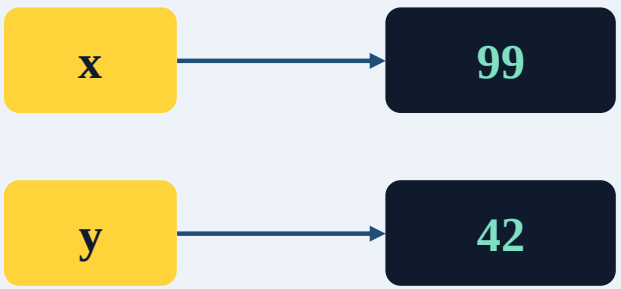
VOCABULARY

Assignment: the equals sign binds the name on its left to the object on its right.

AFTER LINE 2



AFTER LINE 3



KEY IDEA

A variable is a label tied to an object. **y = x** makes both labels point at the **same** object. No copy is made.

UNDER THE HOOD

Line 3 does not change the object 42. It creates the object 99 and moves the label **x** onto it, so **y** still sees 42.

Assignment and naming rules

PYTHON

```
age = 30
first_name = "Grace"
PI = 3.14159   # capitals signal a constant
is_ready = True

x, y = 10, 20   # two assignments in one line
a = b = c = 0   # one object, three names

age = age + 1   # right side first, then store
print(first_name, age, x, y, a, b, c)
```

OUTPUT

```
Grace 31 10 20 0 0 0
```

RULES

- Names use letters, digits and underscores.
- A name cannot start with a digit.
- Names are case sensitive: **age** and **Age** differ.
- Keywords such as **if**, **for** and **class** are reserved.

TIP

Write names in **snake_case** and make them descriptive. **total_price** beats **tp** every time.

WATCH OUT

The equals sign is not a claim that two things are equal. It is a command: evaluate the right side and bind the name on the left.

Numbers: int and float

PYTHON

```
count = 42   # int: whole numbers of any size
price = 19.99   # float: numbers with a decimal point
big = 2 ** 100   # Python integers never overflow
readable = 1_000_000   # underscores help the eye

print(type(count), type(price))
print(big)
print(readable + 1)
print(10 / 4, 10 // 4, 10 % 4)
print(abs(-7.5), round(3.14159, 2), max(3, 9), min(3, 9))
```

OUTPUT

```
<class 'int'> <class 'float'>
1267650600228229401496703205376
1000001
2.5 2 2
7.5 3.14 9 3
```

TWO NUMERIC TYPES

- **int** holds whole numbers with unlimited precision.
- **float** holds real numbers with about 15 to 17 significant digits.

KEY IDEA

Mixing an **int** and a **float** in one expression always produces a **float**.

UNDER THE HOOD

Python also has a built in **complex** type. Write the imaginary part with a **j**, as in **3 + 4j**.

TRY IT YOURSELF

How many digits does **2 ** 1000** have? Combine **len()** and **str()** to find out.

Arithmetic operators

PYTHON

```
a, b = 17, 5
print("a + b  =", a + b)   # addition
print("a - b  =", a - b)   # subtraction
print("a * b  =", a * b)   # multiplication
print("a / b  =", a / b)   # true division: always a float
print("a // b =", a // b)   # floor division: rounds down
print("a % b  =", a % b)   # remainder (modulo)
print("a ** b =", a ** b)   # power
print(divmod(a, b))   # quotient and remainder as a pair
print(-17 // 5, -17 % 5)   # floor goes toward negative infinity
```

OUTPUT

```
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a // b = 3
a % b  = 2
a ** b = 1419857
(3, 2)
-4 3
```

KEY IDEA

/ always returns a float. // drops the fraction by rounding **down**. % gives what is left over.

TIP

Use **n % 2 == 0** to test for even numbers and **i % size** to wrap an index around.

Floats are approximations

PYTHON

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
import math
print(math.isclose(0.1 + 0.2, 0.3))   # compare with a tolerance
print(round(2.675, 2))   # surprise: not 2.68

from decimal import Decimal   # exact decimal arithmetic
print(Decimal("0.1") + Decimal("0.2"))

from fractions import Fraction   # exact rational arithmetic
print(Fraction(1, 3) + Fraction(1, 6))
```

OUTPUT

```
0.30000000000000004
False
True
2.67
0.3
1/2
```

UNDER THE HOOD

Floats are stored in binary. Just as one third has no finite decimal form, one tenth has no finite binary form, so the stored value is very slightly off.

WATCH OUT

Never test floats with ==. Use **math.isclose()** or compare the difference against a small tolerance.

TIP

Use **Decimal** for money and **Fraction** for exact ratios. Always build a **Decimal** from a string, not from a float.

Strings: creating text

PYTHON

```
single = 'Hello'
double = "It's easy"   # one kind of quote inside the other
multi = """Line one
Line two"""
escaped = "Tab:\tDone\nNew line"
raw = r"C:\new\table"   # raw string: backslashes stay
print(double, multi, sep="\n")
print(escaped)
print(raw)
print(len(single), single + " there", "ha" * 3)
```

OUTPUT

```
It's easy
Line one
Line two
Tab:	Done
New line
C:\new\table
5 Hello there hahaha
```

FOUR WAYS TO WRITE TEXT

- Single or double quotes for one line.
- Triple quotes for text that spans lines.
- Escape sequences such as **\n** (new line) and **\t** (tab).
- An **r** prefix switches escapes off.

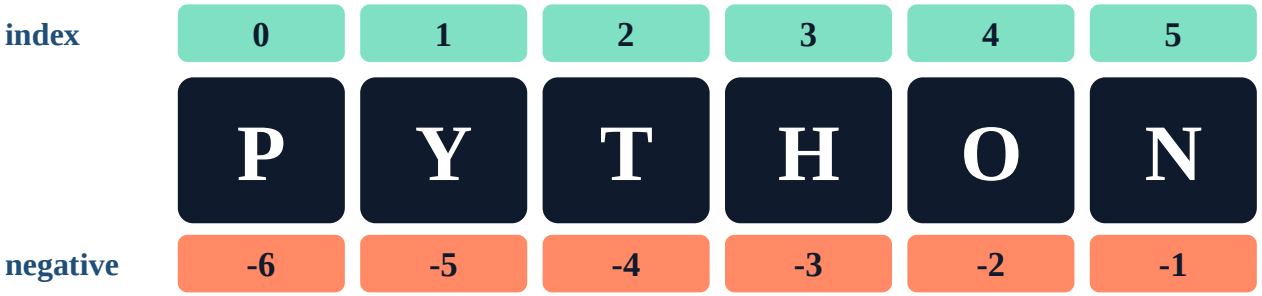
KEY IDEA

+ joins strings, ***** repeats them and **len()** counts their characters.

VOCABULARY

String: an ordered, immutable sequence of Unicode characters. Its type is **str**.

Indexing and slicing



KEY IDEA

s[start:stop:step] begins at **start**, ends **before** **stop** and jumps by **step**. Every part is optional.

TIP

Counting starts at zero. Negative indexes count back from the end, so the last character is always one step before the end.

PYTHON

```
s = "PYTHON"
print(s[0], s[5], s[-1])   # single characters
print(s[0:2])   # start at 0, stop before 2
print(s[2:])   # from 2 to the end
print(s[:4])   # from the start, stop before 4
print(s[-3:])   # the last three
print(s[::2])   # every second character
print(s[::-1])   # step of minus one: reversed
```

OUTPUT

```
P N N
PY
THON
PYTH
HON
PTO
NOHTYP
```

WATCH OUT

An index past the end raises **IndexError**. A slice past the end is simply cut short.

TRY IT YOURSELF

Print the middle two letters of the word in two different ways.

String methods

PYTHON

```
text = "  Python is Fun  "
print(text.strip())   # trim outer spaces
print(text.lower(), "|", text.upper())
print(text.strip().replace("Fun", "powerful"))
print(text.split())   # string to list of words
print(", ".join(["a", "b", "c"]))   # list back to one string
print("is" in text, text.find("is"), text.count("n"))
print("report.pdf".endswith(".pdf"), "42".isdigit())
print("python".capitalize(), "|", "hello world".title())
```

OUTPUT

```
Python is Fun
  python is fun   |   PYTHON IS FUN
Python is powerful
['Python', 'is', 'Fun']
a, b, c
True 9 2
True True
Python | Hello World
```

VOCABULARY

Method: a function that belongs to an object. Call it with a dot: **text.upper()**.

KEY IDEA

No string method changes the original text. Each one returns a **new** string that you must store or use.

TIP

Chain methods from left to right: **text.strip().lower()** trims first and then lowers the result.

TRY IT YOURSELF

Turn **'2026/10/06'** into the list **['2026', '10', '06']** with one method call.

Strings are immutable

NOT ALLOWED

```
word = "jello"
word[0] = "h"   # try to change one letter
```

ERROR

TypeError: 'str' object does not support item assignment

BUILD A NEW STRING

```
word = "jello"
word = "h" + word[1:]   # new string, same name
shout = word.upper()   # word is unchanged
print(word, shout)
```

OUTPUT

hello HELLO

KEY IDEA

Once created, a string can never be altered. To change text you build a new string and rebind the name.

WATCH OUT

A bare **name.strip()** on its own line does nothing useful. Write **name = name.strip()** to keep the result.

UNDER THE HOOD

Immutability lets Python share strings safely and use them as dictionary keys. Module 5 returns to this idea.

f strings: formatting output

PYTHON

```
name, score, total = "Lin", 47, 60
pct = score / total

print(f"{name} scored {score} out of {total}")
print(f"Percentage: {pct:.1%}")
print(f"Pi to 3 places: {3.14159265:.3f}")
print(f"Big number: {1234567.891:,.2f}")
print(f"[{name:<8}] [{name:>8}] [{name:^8}]")
print(f"{score = }")
print(f"Next year: {2026 + 1}, shout: {name.upper()}")
```

OUTPUT

```
Lin scored 47 out of 60
Percentage: 78.3%
Pi to 3 places: 3.142
Big number: 1,234,567.89
[Lin     ] [     Lin] [  Lin   ]
score = 47
Next year: 2027, shout: LIN
```

KEY IDEA

Put an **f** before the opening quote. Anything inside curly braces is evaluated and inserted into the text.

FORMAT CODES AFTER THE COLON

- **.3f** fixes three decimal places.
- **,** adds a thousands separator.
- **.1%** multiplies by 100 and adds a percent sign.
- **<**, **>** and **^** align inside a given width.
- A trailing **=** prints the expression and its value.

Booleans and None

PYTHON

```
is_adult = 20 >= 18
print(is_adult, type(is_adult))
print(True + True)   # bool is a subtype of int

result = None   # a deliberate "nothing here"
print(result is None)

def greet():
    print("hi")   # no return statement

value = greet()
print(value)
```

OUTPUT

```
True <class 'bool'>
2
True
hi
None
```

BOOLEANS

The type **bool** has exactly two values, **True** and **False**, both written with a capital letter. Comparisons produce them.

NONE

None is a single special object that means no value. A function without a **return** statement hands back **None**.

TIP

Test for **None** with **is None** or **is not None**, never with ==.

Type conversion

PYTHON

```
print(int("42") + 1)   # str to int
print(float("3.5") * 2)   # str to float
print(str(42) + "!")   # int to str
print(int(3.99))   # float to int: cuts, no rounding
print(round(3.99))   # round to the nearest int
print(bool(0), bool(""), bool("no"))
print(list("abc"), tuple([1, 2]))
print(int("forty"))   # impossible: raises ValueError
```

OUTPUT

```
43
7.0
42!
3
4
False False True
['a', 'b', 'c'] (1, 2)
ValueError: invalid literal for int() with base 10: 'forty'
```

KEY IDEA

Python is strongly typed. It will not silently turn **'5'** into **5**. You convert on purpose by calling the target type.

WATCH OUT

int() truncates toward zero. **int(3.99)** is **3**, and **int('3.99')** is an error because the text is not a whole number.

TRY IT YOURSELF

What does **int(' 12 ')** return? What about **float('1e3')**? Predict first, then test.

Input and output

PYTHON

```
name = input("What is your name? ")
age = int(input("How old are you? "))

print("Hello", name, sep=", ", end="!\n")
print(f"Next year you will be {age + 1}.")
print("a", "b", "c", sep=" | ")
```

A SAMPLE RUN

```
What is your name? Ada
How old are you? 36
Hello, Ada!
Next year you will be 37.
a | b | c
```

KEY IDEA

input() pauses the program, waits for the Enter key and **always returns a string**.

PRINT OPTIONS

- **sep** sets the text placed between values. The default is one space.
- **end** sets the text printed last. The default is a new line.

WATCH OUT

Forgetting **int()** around **input()** is the most common beginner bug: **'36' + 1** raises a **TypeError**.

Quiz: strings and numbers

PREDICT THE OUTPUT

```
a = "5"
b = 2
print(a * b)
print(int(a) * b)
```

ACTUAL OUTPUT

```
55
10
```

QUESTION

What do the two **print** calls show?

OPTIONS

A. **10** and then **10**

B. **55** and then **10**

C. **52** and then **10**

D. A **TypeError**

ANSWER

B. A string times an integer repeats the text. After conversion to **int** the star multiplies.

Module 2 recap

1. A variable is a name bound to an object. Assignment never copies the object.
2. **int** is exact and unlimited. **float** is fast and approximate, so compare with **math.isclose()**.
3. Strings are immutable sequences. Index from zero and slice with **start:stop:step**.
4. String methods return new strings. f strings build formatted text from expressions.
5. **True**, **False** and **None** are single special objects. Test **None** with **is**.
6. Convert types on purpose with **int()**, **float()**, **str()** and **bool()**. **input()** returns text.

Coming next: Module 3 shows how operators combine values into expressions.

03

MODULE 03 BEGINNER

Operators and Expressions

Compare values, combine conditions and learn the order in which Python evaluates an expression.

IN THIS MODULE

1. Comparison operators
2. Logical operators and short circuits
3. Truthy and falsy values
4. Equality versus identity
5. Operator precedence
6. Augmented assignment and the walrus

Comparison operators

PYTHON

```
x = 7
print(x == 7, x != 7)   # equal, not equal
print(x > 5, x <= 6)   # greater, less or equal
print(1 < x < 10)   # chained: x lies between 1 and 10
print("apple" < "banana")   # text compares alphabetically
print("Zebra" < "apple")   # capitals sort before lowercase
print([1, 2, 3] < [1, 2, 4])   # sequences compare item by item
print(1 == 1.0, "1" == 1)
```

OUTPUT

```
True False
True False
True
True
True
True
True False
```

THE SIX COMPARISONS

== != < <= > >=. Each one returns **True** or **False**.

KEY IDEA

= assigns. == compares. Mixing them up is a syntax error inside an **if** and a silent bug elsewhere.

TIP

Chained comparisons read like mathematics. **0 <= index < len(items)** checks both bounds in one expression.

UNDER THE HOOD

Strings compare by the Unicode code point of each character. **ord('Z')** is 90 and **ord('a')** is 97.

Logical operators and short circuits

PYTHON

```
age, has_ticket = 20, False
print(age >= 18 and has_ticket)   # both must be true
print(age >= 18 or has_ticket)   # at least one must be true
print(not has_ticket)   # flips the truth value
# and / or hand back one of their operands
print("" or "guest")   # first truthy value
print("Ada" and "Lovelace")   # last value if all are truthy

# short circuit: the right side runs only when needed
items = []
print(len(items) > 0 and items[0] == "x")
```

OUTPUT

```
False
True
True
guest
Lovelace
False
```

KEY IDEA

and stops at the first falsy operand. **or** stops at the first truthy operand. The rest is never evaluated.

TIP

name = user_input or 'guest' supplies a default whenever the input is empty.

IN PLAIN ENGLISH

In the last line Python sees that the list is empty and never touches ***items[0]****, so no error occurs.*

Truthy and falsy values

Kind of value	Counts as False	Counts as True
Numbers	**0**, **0.0**, **0j**	Every other number, including negative ones
Strings	The empty string **''**	Any text with at least one character, even **' '**
Containers	**[]**, **()**, **{}**, **set()**, **range(0)**	Any container that holds at least one item
Special objects	**None** and **False**	**True**
Your own classes	**__bool__** returns **False** or **__len__** returns **0**	Every object by default

KEY IDEA

Any object can stand where a condition is expected. Python asks it for its truth value.

TIP

Write **if items:** in place of **if len(items) > 0:**. It is shorter and works for every container.

WATCH OUT

if x: also rejects zero. When zero is a valid answer, test **if x is not None:**.

Equality versus identity

== COMPARES VALUES

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)   # same content
print(a is b)   # two separate objects
```

OUTPUT

```
True
False
```

is COMPARES IDENTITY

```
a = [1, 2, 3]
c = a   # a second name for one object
print(c is a)
c.append(4)
print(a)
```

OUTPUT

```
True
[1, 2, 3, 4]
```

KEY IDEA

== asks whether two objects hold equal values. **is** asks whether two names refer to the very same object.

TIP

Reserve **is** for the singletons **None**, **True** and **False**. Use == for everything else.

UNDER THE HOOD

id(obj) returns a number that is unique to an object while it lives. **a is b** is the same test as **id(a) == id(b)**.

Operator precedence

Order	Operators	Example	Evaluates to
1 (first)	Parentheses **( )**	**(2 + 3) * 4**	**20**
2	Power ****	**2 ** 3 ** 2**	**512**, grouped from right to left
3	*** / // %**	**10 + 6 / 2**	**13.0**
4	Addition and subtraction	**2 + 3 * 4**	**14**
5	Comparisons, **in**, **is**	**1 + 1 == 2**	**True**
6	**not**	**not 1 == 2**	**True**
7	**and**	**True or False and False**	**True**
8 (last)	**or**	**a or b and c**	The same as **a or (b and c)**

TIP

You do not need to memorise the whole table. When an expression is not obvious at a glance, add parentheses.

WATCH OUT

A sign in front of a number binds less tightly than power, so the square of minus three needs parentheses around the minus three.

Augmented assignment and the walrus

PYTHON

```
total = 10
total += 5   # the same as total = total + 5
total *= 2
total -= 4
total //= 3
print(total)

words = ["a", "b"]
words += ["c"]   # extends the list in place
print(words)
# walrus operator: assign inside an expression
data = [4, 8, 15, 16, 23, 42]
if (n := len(data)) > 5:
    print(f"The list is long: {n} items")
```

OUTPUT

```
8
['a', 'b', 'c']
The list is long: 6 items
```

AUGMENTED ASSIGNMENT

Every arithmetic operator has a short form that updates a variable: **+=**, ***=**, **//=**, **%=**, ****=** and the rest.

THE WALRUS

:= assigns a value **and** hands it back, so you can name a result in the middle of a condition.

WATCH OUT

Python has no **++** operator. Write **count += 1**.

Quiz: what do and and or return

PREDICT THE OUTPUT

```
x = 0
y = x or 10
z = x and 10
print(y, z)
```

ACTUAL OUTPUT

```
10 0
```

QUESTION

What does this program print?

OPTIONS

A. True False

B. 10 0

C. 10 10

D. 0 10

ANSWER

B. **or** returns the first truthy operand, which is 10. **and** returns the first falsy operand, which is 0.

Module 3 recap

1. Comparisons return **True** or **False** and can be chained: **1 < x < 10**.
2. **and**, **or** and **not** combine conditions. **and** and **or** short circuit and return an operand.
3. Empty containers, zero, the empty string and **None** are falsy. Everything else is truthy.
4. **==** compares values. **is** compares identity and belongs with **None**.
5. Precedence runs from power through arithmetic and comparison to **not**, **and**, **or**.
6. **+=** and friends update a variable. **:=** assigns inside an expression.

Coming next: Module 4 uses these conditions to make decisions and repeat work.

04

MODULE 04 BEGINNER

Control Flow

Teach a program to choose between paths and to repeat work until a job is done.

IN THIS MODULE

1. Decisions with if, elif and else
2. Pattern matching with match
3. while loops
4. for loops and range
5. break, continue and loop else
6. enumerate, zip and loop patterns

Decisions: if, elif, else

PYTHON

```python
temperature = 23

if temperature >= 30:
    advice = "Stay in the shade"
elif temperature >= 20:
    advice = "Perfect for a walk"
elif temperature >= 10:
    advice = "Take a jacket"
else:
    advice = "Wear a coat"

print(advice)
```

1
2
3

OUTPUT

```
Perfect for a walk
```

LINE BY LINE

1. Python tests the conditions from top to bottom.
2. The first condition that is true wins. Its block runs and every later branch is skipped.
3. **else** catches whatever is left. It is optional and takes no condition.

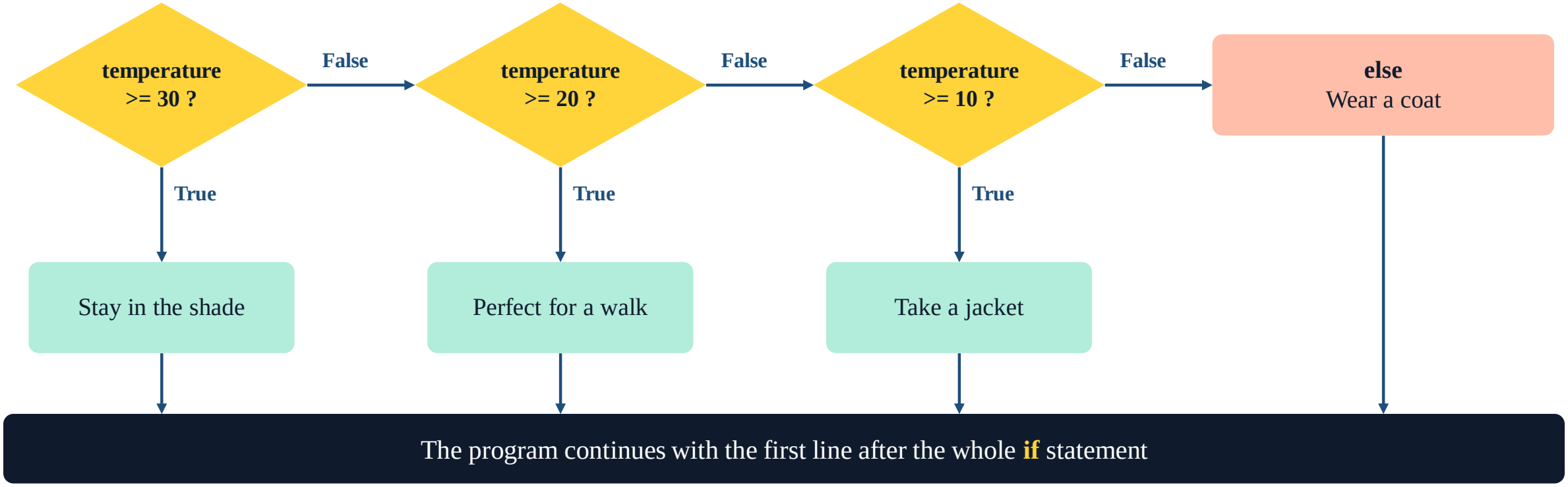
KEY IDEA

Exactly **one** branch of an **if** chain runs, even when several conditions would be true.

WATCH OUT

Order matters. If the test for 20 came before the test for 30, a hot day would be called perfect for a walk.

How Python walks through the branches



KEY IDEA

Follow the arrows: as soon as one diamond answers True, Python runs that block and jumps straight to the bottom bar.

TIP

Draw a diagram like this one on paper whenever a chain of conditions confuses you.

Nested ifs and conditional expressions

DEEPLY NESTED

```
age, is_member = 30, True
if age >= 18:
    if is_member:
        price = 8
    else:
        price = 12
else:
    price = 5
print(price)
```

OUTPUT

```
8
```

FLAT AND CLEAR

```
age, is_member = 30, True
if age < 18:
    price = 5
elif is_member:
    price = 8
else:
    price = 12
status = "adult" if age >= 18 else "minor"
print(price, status)
```

OUTPUT

```
8 adult
```

CONDITIONAL EXPRESSION

a if condition else b chooses between two **values** on a single line. It is an expression, so it can sit on the right of an assignment.

TIP

Flat code is easier to read than nested code. Handle the special case first and let the rest follow.

WATCH OUT

Do not nest one conditional expression inside another. Use a full **if** statement when there are more than two outcomes.

Pattern matching with match

PYTHON

```python
command = "go north"

match command.split():
    case ["quit"]:
        print("Goodbye")
    case ["go", direction]:
        print(f"Walking {direction}")
    case ["take", item] | ["grab", item]:
        print(f"You now carry the {item}")
    case _:
        print("I do not understand")
```

1 2 3 4

OUTPUT

```
Walking north
```

LINE BY LINE

1. **match** takes one value and compares it with each **case** from the top.
2. A bare name inside a pattern **captures** that part of the value.
3. A vertical bar offers alternatives.
4. The underscore is a wildcard that matches anything.

KEY IDEA

match checks the **shape** of data as well as its content. It is far more than a chain of equality tests.

VOCABULARY

Structural pattern matching arrived in Python 3.10. Module 13 explores class patterns and guards.

while loops: repeat while a condition holds

PYTHON

```
countdown = 3
while countdown > 0:
    print(countdown)
    countdown -= 1   # without this line the loop never ends
print("Lift off!")

# keep asking until the answer is valid
replies = ["", "maybe", "yes"]
answer = replies.pop(0)
while answer not in ("yes", "no"):
    answer = replies.pop(0)
print("Final answer:", answer)
```

OUTPUT

```
3
2
1
Lift off!
Final answer: yes
```

IN PLAIN ENGLISH

While the countdown is above zero: show it, then lower it by one. When the test fails, carry on below the loop.

KEY IDEA

A **while** loop tests its condition **before** every round. Something inside the body must eventually make the condition false.

WATCH OUT

A loop whose condition never becomes false runs forever. Press Ctrl and C in the terminal to stop a runaway program.

for loops and range

PYTHON

```
for letter in "abc":   # any sequence works
    print(letter, end=" ")
print()
for n in range(5):   # 0 up to, not including, 5
    print(n, end=" ")
print()
for n in range(2, 11, 3):   # start, stop, step
    print(n, end=" ")
print()
total = 0
for price in [4.5, 3.0, 12.25]:
    total += price
print("Total:", total)
```

OUTPUT

```
a b c
0 1 2 3 4
2 5 8
Total: 19.75
```

KEY IDEA

A **for** loop takes the items of a collection one at a time and binds each to the loop variable.

RANGE

- **range(stop)** counts from zero.
- **range(start, stop)** sets the first value.
- **range(start, stop, step)** also sets the jump.
- The stop value itself is never produced.

TIP

Loop over the items directly. **for price in prices:** is clearer than indexing with **range(len(prices))**.

break, continue and the loop else

PYTHON

```python
for n in [3, 8, 0, 5]:
    if n == 0:
        continue   # skip the rest of this round
    if n > 7:
        print("big number, moving on")
        continue
    print(10 / n)
for candidate in [9, 15, 22, 31]:
    if candidate % 2 == 0:
        print("first even:", candidate)
        break   # leave the loop at once
else:
    print("no even number found")
```

OUTPUT

```
3.3333333333333335
big number, moving on
2.0
first even: 22
```

THREE TOOLS

- **continue** jumps to the next round.
- **break** ends the loop immediately.
- A loop **else** block runs only when the loop finished **without** a **break**.

IN PLAIN ENGLISH

Search the candidates. Stop at the first even one. If the search runs out, report that nothing was found.

TRY IT YOURSELF

Remove 22 from the list and run the program again. Which line prints now, and why?

enumerate and zip

PYTHON

```python
names = ["Ada", "Grace", "Linus"]
scores = [95, 88, 72]
for i, name in enumerate(names, start=1):
    print(i, name)
for name, score in zip(names, scores):
    print(f"{name}: {score}")
print(list(zip(names, scores)))
print(dict(zip(names, scores)))
```

OUTPUT

```
1 Ada
2 Grace
3 Linus
Ada: 95
Grace: 88
Linus: 72
[('Ada', 95), ('Grace', 88), ('Linus', 72)]
{'Ada': 95, 'Grace': 88, 'Linus': 72}
```

ENUMERATE

enumerate() pairs each item with a running counter, so you never manage an index variable by hand.

ZIP

zip() walks through several sequences side by side and yields one tuple per position. It stops at the shortest input.

TIP

Pass **strict=True** to **zip()** when the inputs must have equal lengths. Python then raises an error on a mismatch.

Nested loops and the trace table

PYTHON

```
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end=" ")
    print()
```

OUTPUT

```
1 2 3
2 4 6
3 6 9
```

KEY IDEA

The inner loop runs **completely** for every single round of the outer loop: 3 rounds times 3 rounds gives 9 steps.

TRACE TABLE

step	row	col	row * col
1	1	1	1
2	1	2	2
3	1	3	3
4	2	1	2
5	2	2	4
6	2	3	6
7	3	1	3
8	3	2	6
9	3	3	9

TIP

A trace table tracks every variable by hand, one step per row. It is the best tool for understanding any loop.

Four loop patterns you will use forever

1 Accumulate

Start from a neutral value and update it in every round.

```
total = 0
for n in [4, 8, 15]:
    total += n
```

2 Count

Add one each time a condition holds.

```
vowels = 0
for ch in "education":
    if ch in "aeiou":
        vowels += 1
```

3 Search

Stop as soon as you have found what you need.

```
for word in ["tea", "milk", "jam"]:
    if word.startswith("m"):
        break
```

4 Best so far

Remember the champion and replace it when a better one appears.

```
best = None
for n in [7, 42, 19]:
    if best is None or n > best:
        best = n
```

Quiz: continue in a loop

PREDICT THE OUTPUT

```
total = 0
for n in range(1, 6):
    if n % 2 == 0:
        continue
    total += n
print(total)
```

ACTUAL OUTPUT

9

QUESTION

What is printed?

OPTIONS

A. 15

B. 9

C. 6

D. 5

ANSWER

B. The even numbers 2 and 4 are skipped, so the loop adds 1, 3 and 5.

Module 4 recap

1. **if**, **elif** and **else** run exactly one branch: the first whose condition is true.
2. **a if condition else b** picks a value. **match** compares the shape of data against patterns.
3. **while** repeats as long as its condition holds. Make sure the body can end it.
4. **for** visits every item of a collection. **range()** produces counting sequences.
5. **break** leaves a loop, **continue** skips a round and the loop **else** runs when no **break** happened.
6. **enumerate()** adds a counter and **zip()** pairs sequences. A trace table explains any loop.

Coming next: Module 5 introduces lists, tuples, dictionaries and sets.

05

MODULE 05 BEGINNER

Data Structures

Store many values at once and choose the right container for every job.

IN THIS MODULE

1. Lists and their methods
2. Aliases and copies
3. Tuples and unpacking
4. Dictionaries and common patterns
5. Sets and set algebra
6. Comprehensions
7. Sorting and performance

Lists: ordered and mutable

PYTHON

```python
fruits = ["apple", "banana", "cherry"]
print(fruits[0], fruits[-1], len(fruits))
fruits[1] = "blueberry"   # replace an item
fruits.append("date")   # add at the end
fruits.insert(0, "apricot")   # add at a position
print(fruits)
removed = fruits.pop()   # remove and return the last
fruits.remove("apple")   # remove by value
print(removed, fruits)
print("cherry" in fruits, fruits.index("cherry"))
mixed = [1, "two", 3.0, [4, 5]]   # any types, even lists
print(mixed[3][0])
```

OUTPUT

```
apple cherry 3
['apricot', 'apple', 'blueberry', 'cherry', 'date']
date ['apricot', 'blueberry', 'cherry']
True 2
4
```

VOCABULARY

List: an ordered collection that can grow, shrink and change. Write it with square brackets.

KEY IDEA

Lists support the same indexing and slicing as strings, but unlike strings they can be modified in place.

TRY IT YOURSELF

Start with an empty list **[]**, append the numbers 1 to 5 in a loop and print the result.

The essential list methods

Method	What it does	Returns
append(x)	Adds one item at the end	**None**
extend(items)	Adds every item of another collection	**None**
insert(i, x)	Inserts **x** before position **i**	**None**
pop(i)	Removes the item at position **i**, by default the last one	The removed item
remove(x)	Removes the first item equal to **x**	**None**
sort() and **reverse()**	Reorder the list in place	**None**
index(x) and **count(x)**	Find the position of **x** or count its occurrences	An integer
copy() and **clear()**	Make a shallow copy or empty the list	A new list or **None**

WATCH OUT

Methods that change a list in place return **None**. Writing **items = items.sort()** throws your list away.

TIP

sorted(items) and **reversed(items)** leave the original untouched and give you a new result.

Slicing lists

PYTHON

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[1:4])   # positions 1, 2 and 3
print(nums[:2], nums[4:])   # both ends
print(nums[::2])   # every second item
print(nums[::-1])   # a reversed copy
nums[1:3] = [21, 22, 23]   # replace a slice, sizes may differ
print(nums)
del nums[-2:]   # delete the last two items
print(nums)
clone = nums[:]   # a full slice is a shallow copy
print(clone == nums, clone is nums)
```

OUTPUT

```
[20, 30, 40]
[10, 20] [50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
[10, 21, 22, 23, 40, 50, 60]
[10, 21, 22, 23, 40]
True False
```

KEY IDEA

A slice of a list is always a **new** list. Assigning **to** a slice changes the original list in place.

TIP

nums[:] and **nums.copy()** both make a shallow copy of the whole list.

Aliasing: two names, one list

PYTHON

```
a = [1, 2, 3]
b = a   # alias: no copy is made
c = a.copy()   # a real, separate copy
b.append(4)
print(a, b, c)
print(a is b, a is c)
```

1 2 3

OUTPUT

```
[1, 2, 3, 4] [1, 2, 3, 4] [1, 2, 3]
True False
```

WATCH OUT

Changing a list through one name changes it for **every** name bound to that list. This is the source of many surprising bugs.

MEMORY AFTER LINE 4



KEY IDEA

b = a copies the **name tag**, not the list. **a.copy()** builds a second list with the same items.

TIP

Three ways to copy a list: **a.copy()**, **a[:]** and **list(a)**.

Shallow and deep copies

PYTHON

```python
import copy

grid = [[1, 2], [3, 4]]
shallow = copy.copy(grid)   # new outer list, shared rows
deep = copy.deepcopy(grid)   # every level is duplicated

grid[0][0] = 99
print(grid)
print(shallow)   # the inner list is shared
print(deep)   # fully independent
print(shallow[0] is grid[0], deep[0] is grid[0])
```

OUTPUT

```
[[99, 2], [3, 4]]
[[99, 2], [3, 4]]
[[1, 2], [3, 4]]
True False
```

VOCABULARY

Shallow copy: a new container that holds references to the **same** inner objects.

VOCABULARY

Deep copy: a new container whose contents are copied as well, level after level.

WATCH OUT

.copy(), **[:]** and **list()** are all shallow. For nested data that you intend to modify, use **copy.deepcopy()**.

Tuples and unpacking

PYTHON

```python
point = (3, 4)   # tuple: ordered and immutable
single = (5,)   # the comma makes the tuple
x, y = point   # unpack into two names
print(x, y, point[0], len(point))
q, r = divmod(17, 5)   # a function can return a tuple
print(q, r)
a, b = 1, 2
a, b = b, a   # swap without a helper variable
print(a, b)
for name, age in [("Ada", 36), ("Alan", 41)]:
    print(name, age)
point[0] = 10   # tuples cannot be changed
```

OUTPUT

```
3 4 3 2
3 2
2 1
Ada 36
Alan 41
TypeError: 'tuple' object does not support item assignment
```

VOCABULARY

Tuple: a fixed sequence, written with parentheses. Use it for values that belong together, such as a coordinate.

KEY IDEA

Unpacking assigns each item of a sequence to its own name in one statement. The counts on both sides must match.

TIP

Because tuples never change, they can serve as dictionary keys and set members. Lists cannot.

Star unpacking

PYTHON

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first, middle, last)
head, *tail = "hello"
print(head, tail)

nums = [3, 1, 2]
print(*nums)   # spread into arguments
print(*nums, sep=" < ")
merged = [*nums, 10, *range(3)]   # spread inside a new list
print(merged)
_, month, _ = (2026, "October", 6) # underscore: not needed
print(month)
```

OUTPUT

```
1 [2, 3, 4] 5
h ['e', 'l', 'l', 'o']
3 1 2
3 < 1 < 2
[3, 1, 2, 10, 0, 1, 2]
October
```

STAR ON THE LEFT

A starred name on the left of an assignment collects every item that the other names did not take. The result is always a list.

STAR ON THE RIGHT

A star in front of a collection inside a call or a literal spreads its items out one by one.

TIP

By convention a single underscore names a value that you deliberately ignore.

Dictionaries: look up values by key

PYTHON

```python
user = {"name": "Ada", "age": 36}
print(user["name"])   # look up by key
user["email"] = "ada@example.com"   # add a new pair
user["age"] += 1   # update a value
print(user)
print(user.get("phone"))   # missing key: None
print(user.get("phone", "n/a"))   # or your own default
print("age" in user, len(user))
del user["email"]
print(user.pop("age"), user)
print(user["phone"])   # missing key: KeyError
```

OUTPUT

```
Ada
{'name': 'Ada', 'age': 37, 'email': 'ada@example.com'}
None
n/a
True 3
37 {'name': 'Ada'}
KeyError: 'phone'
```

VOCABULARY

Dictionary: a collection of key and value pairs. Each key is unique and leads straight to its value.

KEY IDEA

Square brackets raise **KeyError** for a missing key. **.get()** returns **None** or a default of your choice.

UNDER THE HOOD

Keys must be hashable, which in practice means immutable: strings, numbers and tuples qualify, lists do not.

Looping over dictionaries

PYTHON

```
stock = {"apples": 12, "pears": 0, "plums": 7}
for fruit in stock:   # keys by default
    print(fruit, end=" ")
print()
print(list(stock.values()), sum(stock.values()))
for fruit, count in stock.items():   # pairs, unpacked
    print(f"{fruit}: {count}")
stock.update({"pears": 5, "kiwis": 3})   # merge in place
both = stock | {"figs": 1}   # a new merged dict
print(both)
```

OUTPUT

```
apples pears plums
[12, 0, 7] 19
apples: 12
pears: 0
plums: 7
{'apples': 12, 'pears': 5, 'plums': 7, 'kiwis': 3, 'figs': 1}
```

THREE VIEWS

- **.keys()** gives the keys.
- **.values()** gives the values.
- **.items()** gives **(key, value)** tuples.

KEY IDEA

Dictionaries remember the order in which keys were inserted, so loops visit pairs in that order.

WATCH OUT

Do not add or delete keys while looping over a dictionary. Loop over **list(stock)** when you must change it.

Dictionary patterns: count and group

PYTHON

```
counts = {}
for ch in "mississippi":
    counts[ch] = counts.get(ch, 0) + 1   # count
print(counts)

words = ["apple", "avocado", "banana", "blueberry", "cherry"]
groups = {}
for word in words:
    groups.setdefault(word[0], []).append(word)   # group
print(groups)
print(max(counts, key=counts.get), sorted(groups))
```

OUTPUT

```
{'m': 1, 'i': 4, 's': 4, 'p': 2}
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
i ['a', 'b', 'c']
```

COUNTING

counts.get(ch, 0) + 1 reads the old count, or zero the first time, and stores the new one.

GROUPING

setdefault(key, []) returns the list stored under the key and creates it first when the key is new.

TIP

Counter and **defaultdict** from the **collections** module do the same jobs with less code. Module 16 shows both.

IN PLAIN ENGLISH

The last line asks: which key has the largest count, and which first letters did we meet?

Sets: unique items and set algebra

PYTHON

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a | b)   # union: in either set
print(a & b)   # intersection: in both sets
print(a - b)   # difference: in a but not in b
print(a ^ b)   # symmetric difference: in exactly one
emails = ["x@a.io", "y@a.io", "x@a.io"]
unique = set(emails)   # duplicates vanish
print(len(unique), "y@a.io" in unique)
a.add(9)
a.discard(1)
print(a, {1, 2} <= {1, 2, 3})   # subset test
```

OUTPUT

```
{1, 2, 3, 4, 5}
{3, 4}
{1, 2}
{1, 2, 5}
2 True
{2, 3, 4, 9} True
```

VOCABULARY

Set: an unordered collection of unique, hashable items with very fast membership tests.

TIP

list(set(items)) removes duplicates. **list(dict.fromkeys(items))** removes them **and** keeps the original order.

WATCH OUT

Sets have no positions and no reliable order. An empty set is written **set()**, because **{}** creates an empty dictionary.

The structure of a list comprehension

```
[ n * n | for n in numbers | if n % 2 == 0 ]
```

1. Expression
What goes into the new list

2. Loop
Where the items come from

3. Filter (optional)
Which items are allowed in

THE LOOP IT REPLACES

```
numbers = [1, 2, 3, 4, 5, 6]
result = []
for n in numbers:
    if n % 2 == 0:
        result.append(n * n)
print(result)
```

THE COMPREHENSION

```
numbers = [1, 2, 3, 4, 5, 6]
result = [n * n for n in numbers if n % 2 == 0]
print(result)
```

OUTPUT

```
[4, 16, 36]
```

KEY IDEA

Read a comprehension in this order: the loop, then the filter, then the expression. It always builds a brand new list.

Dictionary, set and generator comprehensions

PYTHON

```
prices = {"tea": 2.5, "cake": 4.0, "juice": 3.2}

with_tax = {item: round(p * 1.2, 2) for item, p in prices.items()}
print(with_tax)
cheap = {item for item, p in prices.items() if p < 3.5}
print(sorted(cheap))   # a set comprehension
lengths = {word: len(word) for word in ["hi", "hello"]}
flipped = {v: k for k, v in lengths.items()}
print(lengths, flipped)
squares = (n * n for n in range(4))   # generator: round brackets
print(sum(squares))
```

OUTPUT

```
{'tea': 3.0, 'cake': 4.8, 'juice': 3.84}
['juice', 'tea']
{'hi': 2, 'hello': 5} {2: 'hi', 5: 'hello'}
14
```

FOUR FLAVOURS

- Square brackets build a list.
- Curly braces with a colon build a dictionary.
- Curly braces without a colon build a set.
- Round brackets build a lazy generator, covered in Module 11.

TIP

If a comprehension no longer fits comfortably on one or two lines, write an ordinary loop. Clarity wins.

Nested data: lists of dictionaries

PYTHON

```python
students = [
    {"name": "Ada", "grades": [90, 95, 100]},
    {"name": "Bob", "grades": [70, 65]},
    {"name": "Cy", "grades": [88, 92, 79]},
]
for s in students:
    avg = sum(s["grades"]) / len(s["grades"])
    print(f"{s['name']}: {avg:.1f}")
top = [s["name"] for s in students if max(s["grades"]) >= 95]
print(top, students[2]["grades"][-1])
matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]
print(flat, matrix[1][2])
```

OUTPUT

```
Ada: 95.0
Bob: 67.5
Cy: 86.3
['Ada'] 79
[1, 2, 3, 4, 5, 6] 6
```

KEY IDEA

Containers can hold other containers. Real data, such as a JSON reply from a web service, is almost always nested like this.

IN PLAIN ENGLISH

Read the chained lookup on line 10 from left to right: take the third student, open the grades and pick the last one.

TIP

In a comprehension with two **for** clauses, read them left to right exactly as you would write nested loops.

Sorting with key functions

PYTHON

```
words = ["banana", "Cherry", "apple", "fig"]
print(sorted(words))   # capitals come first
print(sorted(words, key=str.lower))   # ignore case
print(sorted(words, key=len, reverse=True))
people = [("Ada", 36), ("Bob", 25), ("Cy", 36)]
by_age = sorted(people, key=lambda p: p[1])
print(by_age)   # ties keep their order
print(sorted(people, key=lambda p: (-p[1], p[0])))
words.sort()   # in place, returns None
print(words, min(words, key=len))
```

OUTPUT

```
['Cherry', 'apple', 'banana', 'fig']
['apple', 'banana', 'Cherry', 'fig']
['banana', 'Cherry', 'apple', 'fig']
[('Bob', 25), ('Ada', 36), ('Cy', 36)]
[('Ada', 36), ('Cy', 36), ('Bob', 25)]
['Cherry', 'apple', 'banana', 'fig'] fig
```

KEY IDEA

The **key** argument is a function. Python calls it once per item and sorts by the answers, not by the items themselves.

UNDER THE HOOD

Python sorts are **stable**: items with equal keys keep their original relative order.

VOCABULARY

lambda: a tiny function without a name. **lambda p: p[1]** takes **p** and returns **p[1]**. Module 6 explains it fully.

Choosing the right container

Question	list	tuple	dict	set
Written as	**[1, 2]**	**(1, 2)**	**{'a': 1}**	**{1, 2}**
Keeps order?	Yes	Yes	Yes, insertion order	No
Can change?	Yes	No	Yes	Yes
Duplicates?	Allowed	Allowed	Keys are unique	Never
Reach items by	Position	Position	Key	Membership test only
Best for	A sequence that grows and changes	A fixed record	Fast lookup by name	Uniqueness and set algebra

KEY IDEA

Ask four questions: do I need order, do I need to change it, do I look things up by key and must items be unique?

TIP

When a tuple grows beyond three fields, give the fields names with a dataclass. Module 10 shows how.

How fast are the common operations

Operation	list	dict and set	Reason
Read by position or key	**O(1)**	**O(1)**	Python jumps straight to the slot
Membership test with in	**O(n)**	**O(1)**	A list is scanned item by item. A hash table jumps.
Add at the end	**O(1)**	**O(1)**	Spare capacity is kept ready
Insert or delete at the front	**O(n)**	**O(1)** by key	Every later list item has to shift
Sort	**O(n log n)**	Not applicable	The Timsort algorithm, tuned for real data
len()	**O(1)**	**O(1)**	The size is stored, never recounted

VOCABULARY

Big O notation describes how the work grows with the number of items **n**. **O(1)** stays flat. **O(n)** grows in step with the data.

TIP

Testing membership in a large collection inside a loop? Convert the collection to a **set** first.

Quiz: aliases and new lists

PREDICT THE OUTPUT

```
a = [1, 2, 3]
b = a
b += [4]
c = a + [5]
print(a, c is a)
```

ACTUAL OUTPUT

[1, 2, 3, 4] False

QUESTION

What is printed?

OPTIONS

A. **[1, 2, 3] False**

B. **[1, 2, 3, 4] False**

C. **[1, 2, 3, 4, 5] True**

D. **[1, 2, 3, 4] True**

ANSWER

B. `+=` extends the shared list in place, so **a** sees the change. The plus operator builds a new list.

Module 5 recap

1. Lists are ordered and mutable. Methods that change them in place return **None**.
2. Assignment creates an alias. Copy with **.copy()** and use **deepcopy()** for nested data.
3. Tuples are immutable records. Unpacking and the star spread values across names.
4. Dictionaries map unique keys to values. **.get()**, **.items()** and **setdefault()** cover most needs.
5. Sets hold unique items and support union, intersection and difference.
6. Comprehensions build lists, dictionaries and sets from a loop, an optional filter and an expression.
7. **sorted()** accepts a **key** function and is stable.
8. Lists scan for membership. Dictionaries and sets jump straight to the answer.

Coming next: Module 6 packages code into reusable functions.

06

MODULE 06 INTERMEDIATE

Functions

Name a piece of logic once and reuse it everywhere. Functions are the main tool for taming complexity.

IN THIS MODULE

1. Defining and calling functions
2. Parameters, arguments and defaults
3. Flexible signatures
4. Return values and scope
5. Functions as objects and lambda
6. Closures and recursion

Defining and calling functions

PYTHON

```python
def greet(name):
    """Return a friendly greeting."""
    message = f"Hello, {name}!"
    return message

text = greet("Ada")   # call it and keep the result
print(text)
print(greet("Grace"))   # call it again with new data
print(len(greet("Bo")))   # a call is an expression
```

OUTPUT

```
Hello, Ada!
Hello, Grace!
10
```

VOCABULARY

Function: a named block of code that takes inputs, does one job and hands back a result.

KEY IDEA

Defining a function runs nothing. The body executes only when the function is **called** with parentheses.

IN PLAIN ENGLISH

To greet someone: build the message from their name and give it back to whoever asked.

TIP

Whenever you copy and paste a block of code, stop and turn it into a function.

The structure of a function

PYTHON

```
def rectangle_area(width, height=1.0):          1
    """Return the area of a rectangle."""       2
    area = width * height                       3
    return area                                 4

result = rectangle_area(3, 4)                   5
print(result)
```

OUTPUT

```
12
```

THE FIVE PARTS

1. **Header**: the keyword **def**, a name, the parameters in parentheses and a colon.
2. **Docstring**: one sentence that says what the function does.
3. **Body**: the indented statements. **area** is a local variable.
4. **Return**: ends the call and sends a value back.
5. **Call**: arguments fill the parameters and the result replaces the call.

VOCABULARY

Parameter: a name in the header. **Argument**: the actual value supplied in a call.

Positional and keyword arguments

PYTHON

```
def describe(name, age, city):
    print(f"{name}, {age}, lives in {city}")

describe("Ada", 36, "London")   # by position
describe(name="Alan", city="Wilmslow", age=41)   # by keyword
describe("Grace", city="New York", age=85)   # mixed

describe("Linus", age=55)   # city is missing
```

OUTPUT

```
Ada, 36, lives in London
Alan, 41, lives in Wilmslow
Grace, 85, lives in New York
TypeError: describe() missing 1 required positional argument:
'city'
```

TWO WAYS TO PASS VALUES

- **Positional**: matched from left to right.
- **Keyword**: matched by name, in any order.

KEY IDEA

Positional arguments must come first. After the first keyword argument, every later one must be a keyword as well.

TIP

Use keywords whenever a call would otherwise be a row of unexplained values. **resize(width=800, height=600)** documents itself.

Default parameter values

PYTHON

```python
def power(base, exponent=2):
    return base ** exponent

print(power(5))   # exponent falls back to 2
print(power(5, 3))   # positional override
print(power(base=2, exponent=10))

def connect(host, port=5432, timeout=30, secure=True):
    return f"{host}:{port} timeout={timeout} secure={secure}"

print(connect("db.local"))
print(connect("db.local", 6000, secure=False))
```

OUTPUT

```
25
125
1024
db.local:5432 timeout=30 secure=True
db.local:6000 timeout=30 secure=False
```

KEY IDEA

A parameter with a default value becomes optional. Callers override only what they need.

RULE

Parameters without defaults must come before parameters with defaults in the header.

TRY IT YOURSELF

Add a parameter **greeting** with the default **'Hello'** to the **greet** function from the first slide of this module.

The mutable default trap

THE TRAP

```
def add_item(item, basket=[]):
    basket.append(item)
    return basket

print(add_item("tea"))
print(add_item("milk"))   # surprise!
```

OUTPUT

```
['tea']
['tea', 'milk']
```

THE FIX

```
def add_item(item, basket=None):
    if basket is None:
        basket = []   # a fresh list for each call
    basket.append(item)
    return basket

print(add_item("tea"))
print(add_item("milk"))
```

OUTPUT

```
['tea']
['milk']
```

UNDER THE HOOD

Default values are created **once**, when the **def** statement runs. Every call without a basket shares that single list.

KEY IDEA

Never use a list, dictionary or set as a default. Use **None** and create the container inside the function.

TIP

Immutable defaults such as numbers, strings, tuples and **None** are always safe.

Collecting arguments: *args and **kwargs

PYTHON

```
def total(*numbers):   # extra positionals: a tuple
    return sum(numbers)
print(total(1, 2, 3), total())

def tag(name, **attributes):   # extra keywords: a dict
    parts = [f'{k}="{v}"' for k, v in attributes.items()]
    return f"<{name} {' '.join(parts)}>"
print(tag("img", src="cat.png", width=80))

def report(*args, **kwargs):
    print(args, kwargs)
report(1, 2, mode="fast")
values, options = [1, 2], {"mode": "slow"}
report(*values, **options)   # unpack at the call site
```

OUTPUT

```
6 0
<img src="cat.png" width="80">
(1, 2) {'mode': 'fast'}
(1, 2) {'mode': 'slow'}
```

IN A DEFINITION

One star gathers surplus positional arguments into a tuple. Two stars gather surplus keyword arguments into a dictionary.

IN A CALL

The same symbols do the reverse: they spread a sequence or a dictionary out into separate arguments.

TIP

The names **args** and **kwargs** are only a convention. The stars do the work.

Positional only and keyword only parameters

PYTHON

```python
def clamp(value, /, low, high, *, strict=False):
    if strict and not low <= value <= high:
        raise ValueError("out of range")
    return max(low, min(value, high))

print(clamp(15, 0, 10))
print(clamp(15, low=0, high=10))
print(clamp(5, 0, 10, strict=True))
print(clamp(5, 0, 10, True))   # strict must be named
```

1

OUTPUT

```
10
10
5
TypeError: clamp() takes 3 positional arguments but 4 were given
```

READING THE HEADER

- Parameters **before the slash** can be passed by position only.
- Parameters **between** the slash and the star accept either style.
- Parameters **after the star** must be passed by keyword.

KEY IDEA

Keyword only parameters make calls readable and let you add options later without breaking old code.

UNDER THE HOOD

Many built in functions use the slash. **len(obj)** works, while **len(obj=x)** is rejected.

Return values

PYTHON

```
def stats(values):
    return min(values), max(values), sum(values) / len(values)

low, high, mean = stats([4, 8, 15])   # one tuple, unpacked
print(low, high, mean)

def find_user(users, name):
    for user in users:
        if user == name:
            return user   # leave the function at once
    return None   # an explicit "not found"

print(find_user(["ada", "bob"], "bob"))
print(find_user(["ada", "bob"], "zed"))
```

OUTPUT

```
4 15 9.0
bob
None
```

KEY IDEA

return ends the function immediately and sends one object back to the caller. Code after it does not run.

SEVERAL RESULTS

Separate values with commas to return a tuple. The caller unpacks it into individual names.

WATCH OUT

print shows a value to a human. **return** hands a value to the program. A function that only prints gives back **None**.

Scope: where Python looks for a name

PYTHON

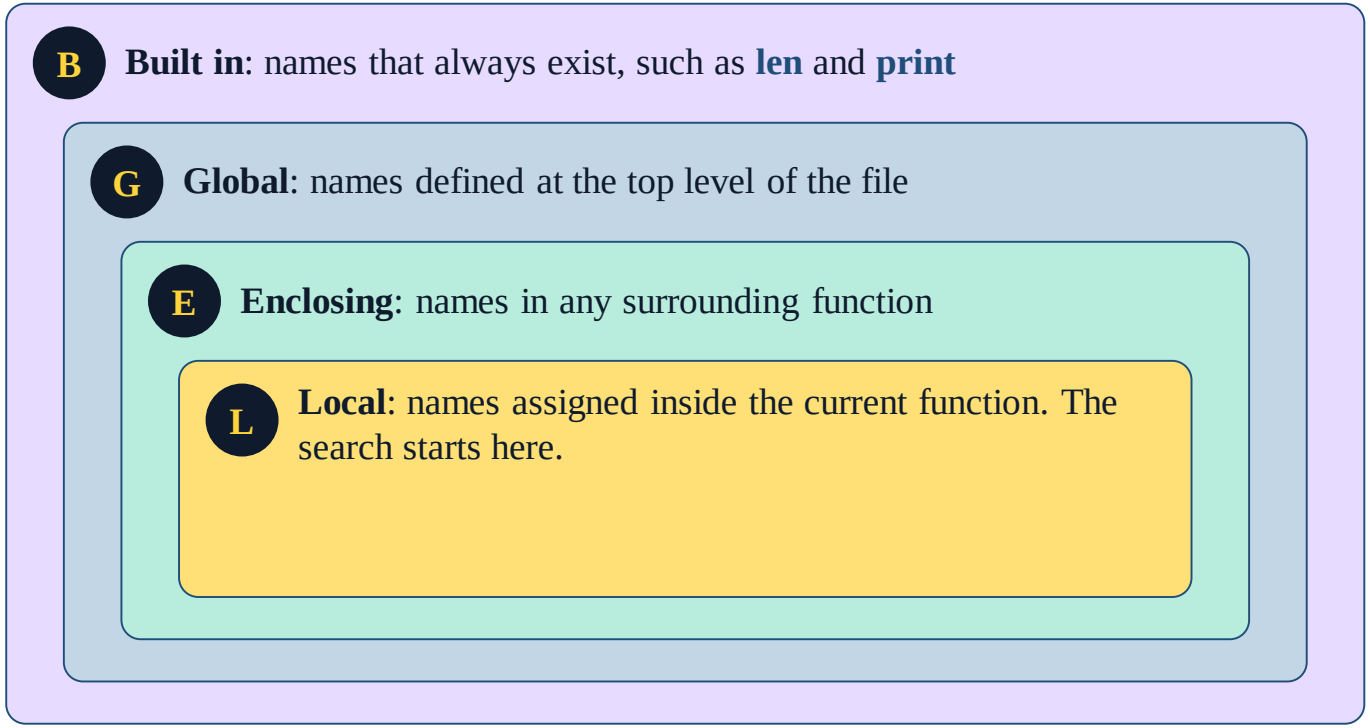
```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()
    print(x)

outer()
print(x)
```

OUTPUT

```
local
enclosing
global
```



KEY IDEA

Python searches **L, E, G, B** in that order and stops at the first match. An inner name hides an outer name of the same spelling.

Changing outer names: global and nonlocal

global

```
counter = 0

def increment():
    global counter   # rebind the module level name
    counter += 1

increment()
increment()
print(counter)
```

OUTPUT

```
2
```

nonlocal

```
def make_counter():
    count = 0
    def step():
        nonlocal count   # rebind the enclosing name
        count += 1
        return count
    return step

step = make_counter()
print(step(), step(), step())
```

OUTPUT

```
1 2 3
```

KEY IDEA

Reading an outer name always works. **Assigning** to it creates a new local name unless you declare **global** or **nonlocal** first.

WATCH OUT

Global variables that many functions modify make programs hard to reason about. Prefer parameters and return values.

Functions are objects

PYTHON

```
def shout(text):
    return text.upper() + "!"

yell = shout   # a second name, no call
print(yell("hello"))
print(shout.__name__, type(shout).__name__)

def apply_twice(func, value):   # a function as an argument
    return func(func(value))

print(apply_twice(shout, "hi"))
actions = {"loud": shout, "quiet": str.lower}
print(actions["quiet"]("HELLO"))
```

OUTPUT

```
HELLO!
shout function
HI!!
hello
```

KEY IDEA

A function is an ordinary object. You can bind it to another name, store it in a container, pass it as an argument and return it.

WITH OR WITHOUT PARENTHESES

shout is the function itself. **shout('hi')** is the result of calling it.

VOCABULARY

Higher order function: a function that takes or returns another function. **sorted**, **map** and **apply_twice** are examples.

Lambda expressions

PYTHON

```
square = lambda x: x * x   # like a one line def
print(square(7))

nums = [5, 2, 8, 1]
print(list(map(lambda n: n * 10, nums)))
print(list(filter(lambda n: n > 2, nums)))
print(sorted(nums, key=lambda n: -n))
print(max(["kiwi", "fig", "banana"], key=len))

# a comprehension is often clearer than map and filter
print([n * 10 for n in nums if n > 2])
```

OUTPUT

```
49
[50, 20, 80, 10]
[5, 8]
[8, 5, 2, 1]
banana
[50, 80]
```

SHAPE OF A LAMBDA

lambda parameters: expression. It has no name, no statements and no **return** keyword. The value of the expression is the result.

TIP

Lambdas shine as short **key** functions. For anything longer than one simple expression, write a normal **def**.

WATCH OUT

Assigning a lambda to a name, as on line 1, is shown only for teaching. Style guides prefer **def** in that case.

Closures: functions that remember

PYTHON

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor   # factor lives in the outer call
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(10), triple(10))
print(double.__closure__[0].cell_contents)

def make_logger(prefix):
    return lambda msg: print(f"[{prefix}] {msg}")

warn = make_logger("WARN")
warn("disk almost full")
```

OUTPUT

```
20 30
2
[WARN] disk almost full
```

VOCABULARY

Closure: an inner function together with the variables it captured from the scope where it was created.

KEY IDEA

make_multiplier has finished, yet **double** still remembers that **factor** is 2. Each call creates a separate memory.

UNDER THE HOOD

Captured variables are stored in cells that hang on the function attribute **__closure__**.

Recursion: a function that calls itself

PYTHON

```python
def factorial(n):
    if n <= 1:   # base case: stop here
        return 1
    return n * factorial(n - 1)   # recursive case

print(factorial(5))

def count_files(tree):
    total = 0
    for item in tree:
        if isinstance(item, list):
            total += count_files(item)   # a folder: go deeper
        else:
            total += 1
    return total
print(count_files(["a.txt", ["b.txt", ["c.txt", "d.txt"]], "e.txt"]))
```

OUTPUT

```
120
5
```

TWO INGREDIENTS

- A **base case** that answers directly.
- A **recursive case** that calls the function on a smaller problem.

KEY IDEA

Recursion fits data that contains smaller copies of itself: folders inside folders, lists inside lists, trees.

WATCH OUT

Without a reachable base case the calls never stop and Python raises **RecursionError**.

The call stack during recursion

PYTHON

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

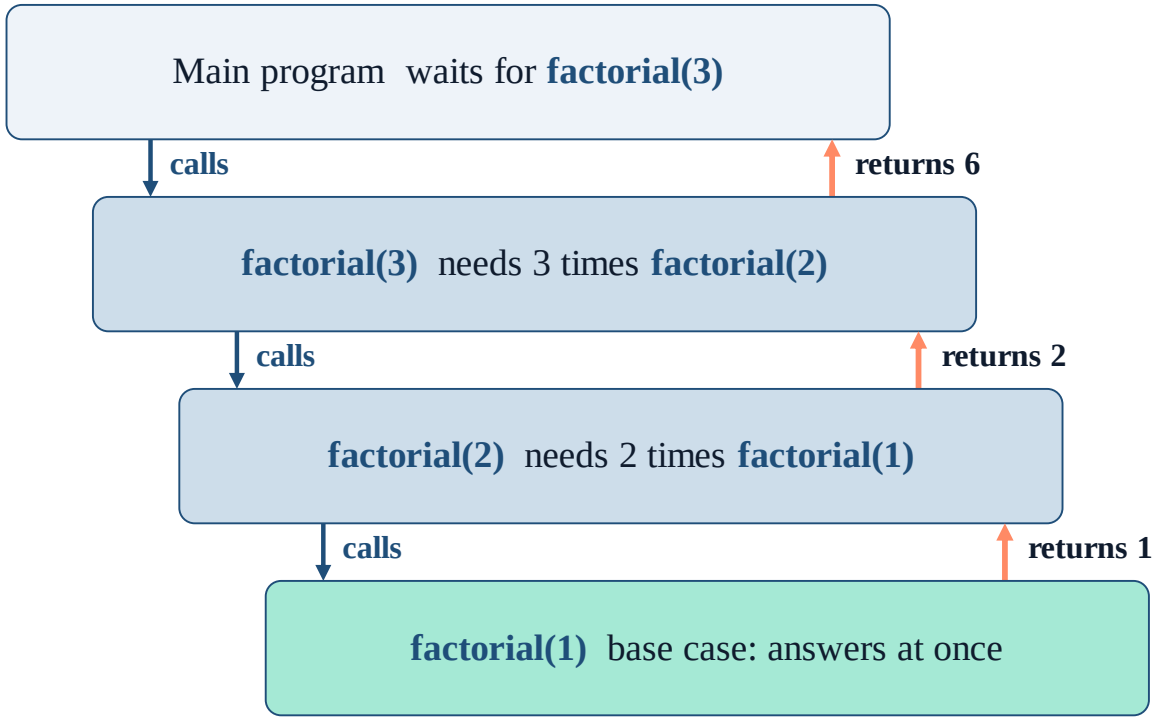
print(factorial(3))
```

OUTPUT

```
6
```

VOCABULARY

Call stack: the pile of unfinished calls. The newest call sits on top and must finish first.



UNDER THE HOOD

Every call gets its own frame with its own copy of **n**. Frames pile up on the way down and are removed, one by one, as the answers travel back up.

Docstrings and type hints

PYTHON

```
def bmi(weight_kg: float, height_m: float) -> float:
    """Return the body mass index.

    Args:
        weight_kg: body weight in kilograms.
        height_m: height in metres.
    """
    return weight_kg / height_m ** 2

print(round(bmi(70, 1.75), 1))
print(bmi.__annotations__)
```

OUTPUT

```
22.9
{'weight_kg': <class 'float'>, 'height_m': <class 'float'>,
'return': <class 'float'>}
```

TYPE HINTS

A colon after a parameter and an arrow before the final colon state the expected types. They document intent and power editor help.

KEY IDEA

Python does **not** enforce type hints at run time. External tools such as mypy check them. Module 13 covers typing in depth.

TIP

Call **help(bmi)** in the shell to see the signature and the docstring nicely formatted.

Quiz: default arguments

PREDICT THE OUTPUT

```
def add(x, items=[]):
    items.append(x)
    return len(items)

print(add("a"), add("b"), add("c", []))
```

ACTUAL OUTPUT

1 2 1

QUESTION

What is printed?

OPTIONS

A. 1 1 1

B. 1 2 3

C. 1 2 1

D. A **TypeError**

ANSWER

C. The first two calls share the single default list. The third call brings its own empty list.

Module 6 recap

1. **def** creates a function. Nothing in the body runs until the function is called.
2. Arguments match parameters by position or by keyword. Defaults make parameters optional.
3. Use **None**, never a mutable object, as a default value.
4. ***args** and ****kwargs** collect surplus arguments. A slash and a star restrict how parameters are passed.
5. Names are found by the LEGB rule. **global** and **nonlocal** allow rebinding outer names.
6. Functions are objects: pass them, store them and return them. A closure remembers its birth scope.
7. A lambda is a one expression function, ideal as a **key**.
8. Recursion needs a base case and a smaller subproblem. Each call has its own stack frame.

Coming next: Module 7 organises functions into modules and packages.

07

MODULE 07 INTERMEDIATE

Modules and Packages

Split programs into files, reuse the standard library and install code written by others.

IN THIS MODULE

1. Import statements
2. A tour of the standard library
3. Writing your own module
4. Packages and project layout
5. Virtual environments and pip
6. How import finds code

Importing modules

PYTHON

```
import math   # the whole module
print(math.sqrt(16), math.pi)
from math import floor, ceil   # selected names only
print(floor(2.7), ceil(2.1))
import statistics as stats   # a shorter alias
print(stats.mean([2, 4, 9]))

from datetime import date
print(date(2026, 10, 6).strftime("%d %B %Y"))

import sys
print(sys.version_info.major, "math" in sys.modules)
```

OUTPUT

```
4.0 3.141592653589793
2 3
5
06 October 2026
3 True
```

VOCABULARY

Module: a single **.py** file. Its functions, classes and variables become attributes of the module object.

THREE IMPORT FORMS

- **import m** keeps names behind the prefix **m.**
- **from m import a, b** brings chosen names in directly.
- **import m as alias** renames the module locally.

WATCH OUT

Avoid **from m import ***. It floods your namespace and hides where each name came from.

A tour of the standard library

Python ships with more than two hundred modules. Learn that these exist before you write the same thing yourself or install a package.

pathlib, os, shutil

Paths, folders, copying and deleting files.

sys, argparse

Interpreter settings and command line arguments.

math, statistics, random

Numeric functions, averages and random numbers.

datetime, time, zoneinfo

Dates, clocks, durations and time zones.

json, csv, sqlite3

Read and write common data formats and databases.

re, string, textwrap

Regular expressions and text utilities.

collections, itertools, functools

Smarter containers, iterator tools and function helpers.

logging, unittest, pdb

Record events, test code and debug step by step.

TIP

The official documentation at docs.python.org has a page for every module. Skim the index once so that you know what is available.

TRY IT YOURSELF

Open the shell and run **import this**, then **import antigravity**. The standard library has a sense of humour.

Writing your own module

FILE: shapes.py

```
"""Area helpers."""
PI = 3.14159

def circle_area(r):
    return PI * r * r

if __name__ == "__main__":
    # runs only when this file is executed directly
    print(circle_area(1))
```

FILE: main.py

```
import shapes
from shapes import circle_area

print(shapes.PI)
print(circle_area(2))
print(shapes.__name__)
```

RUN WITH: python shapes.py

```
3.14159
```

RUN WITH: python main.py

```
3.14159
12.56636
shapes
```

KEY IDEA

Any **.py** file is a module. Importing it runs the file once, top to bottom, and gives you access to every name it defines.

UNDER THE HOOD

__name__ equals **'__main__'** only in the file you launch. In an imported file it holds the module name.

Packages: folders of modules

FOLDER STRUCTURE

```
shop/
    __init__.py   # marks the folder as a package
    cart.py
    pricing.py
    payments/
        __init__.py
        card.py
main.py
```

IMPORTS IN main.py

```
import shop.cart
from shop import pricing
from shop.payments.card import charge
from shop.pricing import apply_tax as tax

# inside shop/cart.py a relative import
# reaches a sibling module:
# from . import pricing
```

VOCABULARY

Package: a folder that contains modules and usually an **__init__.py** file. Packages can nest.

DOTTED NAMES

Each dot in an import steps one level down the folder tree: **shop.payments.card** is the file **shop/payments/card.py**.

TIP

__init__.py runs when the package is first imported. Keep it short, or leave it empty.

Virtual environments and pip

TERMINAL

```
# create an isolated environment in the folder .venv
python -m venv .venv

# activate it on macOS and Linux
source .venv/bin/activate
# activate it in Windows PowerShell
.venv\Scripts\Activate.ps1

# install, inspect and record packages
pip install requests
pip list
pip freeze > requirements.txt

# rebuild the same environment on another machine
pip install -r requirements.txt
deactivate
```

VOCABULARY

Virtual environment: a private folder with its own Python and its own installed packages.

KEY IDEA

Create one environment per project. Projects can then depend on different versions of the same library without conflict.

VOCABULARY

pip: the package installer. It downloads libraries from the Python Package Index, known as PyPI.

WATCH OUT

Never commit the **.venv** folder to version control. Commit **requirements.txt** or **pyproject.toml** instead.

How import finds a module

The statement **import shapes** triggers the same four steps every time.

1 Check the cache

Python looks in **sys.modules**. A module that was already imported is reused instantly.

2 Search

Built in modules come first, then every folder listed in **sys.path**, in order.

3 Load and run

The file is compiled and executed once, top to bottom, inside a new module object.

4 Bind

The module is stored in the cache and bound to a name in the importing file.

UNDER THE HOOD

The first entry of **sys.path** is the folder of the script you launched. Installed packages live in a separate folder that pip manages.

WATCH OUT

Do not give your own file the name of a standard module. A file named **random.py** next to your script hides the real **random**.

The structure of a modern project

PROJECT FOLDER

```
weather_app/
    pyproject.toml    # metadata and dependencies
    README.md
    src/
        weather/
            __init__.py
            api.py
            cli.py
    tests/
        test_api.py
    .venv/    # never committed
```

FILE: pyproject.toml

```
[project]
name = "weather"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["requests"]

[project.scripts]
weather = "weather.cli:main"
```

ONE FILE TO DESCRIBE IT ALL

pyproject.toml is the standard place for the project name, version, dependencies and tool settings.

TIP

Keeping code under **src/** and tests under **tests/** prevents accidental imports of files that are not installed.

TRY IT YOURSELF

Run **pip install .** inside the project folder. The command **weather** then works from any terminal.

Module 7 recap

1. A module is a file. A package is a folder of modules. Both are imported with **import**.
2. **import m**, **from m import x** and **import m as alias** are the three import forms.
3. The standard library covers files, math, dates, text, data formats, testing and more.
4. **if __name__ == '__main__':** separates code to run from code to import.
5. A virtual environment isolates the packages of one project. **pip** installs them from PyPI.
6. Imports are cached in **sys.modules** and searched along **sys.path**.

Coming next: Module 8 reads and writes files and common data formats.

08

MODULE 08 INTERMEDIATE

Files and Data Formats

Make data outlive the program: read and write text, paths, CSV, JSON and raw bytes.

IN THIS MODULE

1. Opening files safely with with
2. File modes and reading strategies
3. Paths with pathlib
4. CSV and JSON
5. Binary data and pickle
6. Text encodings

Reading and writing text files

PYTHON

```python
with open("notes.txt", "w", encoding="utf8") as f:
    f.write("First line\n")
    f.write("Second line\n")
    print("Third line", file=f)   # print can target a file

with open("notes.txt", encoding="utf8") as f:
    content = f.read()   # whole file as one string
print(content, end="")

with open("notes.txt", encoding="utf8") as f:
    for number, line in enumerate(f, start=1):   # line by line
        print(number, line.strip())
```

OUTPUT

```
First line
Second line
Third line
1 First line
2 Second line
3 Third line
```

LINE BY LINE

1. **open()** returns a file object. **with** closes it for you, even if an error occurs.
2. **write()** adds no line break. You supply the **\n** yourself.
3. Without a mode, the file opens for reading.
4. A file object is iterable. Looping over it yields one line at a time and uses very little memory.

WATCH OUT

Always pass **encoding**. Without it Python uses a platform default, and the same file may read differently on another computer.

File modes at a glance

Mode	Meaning	If the file exists	If the file is missing
'r'	Read (the default)	Opens at the start	Raises **FileNotFoundError**
'w'	Write	Erases the content first	Creates the file
'a'	Append	Writes after the last byte	Creates the file
'x'	Exclusive creation	Raises **FileExistsError**	Creates the file
'r+'	Read and write	Opens at the start, keeps content	Raises **FileNotFoundError**
add 'b'	Binary, as in **'rb'** or **'wb'**	Works with **bytes**, no decoding	Follows the base mode

WATCH OUT

Mode **'w'** destroys the existing content the instant the file is opened, before you write a single character.

TIP

Use **'x'** when overwriting would be a disaster. It refuses to touch an existing file.

VOCABULARY

Text mode decodes bytes into **str**. **Binary mode** hands you the raw **bytes**.

More ways to read and write

PYTHON

```
lines = ["alpha\n", "beta\n", "gamma\n"]
with open("data.txt", "w", encoding="utf8") as f:
    f.writelines(lines)
with open("data.txt", "a", encoding="utf8") as f:   # append
    f.write("delta\n")

with open("data.txt", encoding="utf8") as f:
    first = f.readline()   # one line
    rest = f.readlines()   # remaining lines as a list
print(repr(first), rest)

with open("data.txt", encoding="utf8") as f:
    words = f.read().splitlines()   # no trailing line breaks
print(words, f.closed)
```

OUTPUT

```
'alpha\n' ['beta\n', 'gamma\n', 'delta\n']
['alpha', 'beta', 'gamma', 'delta'] True
```

CHOOSE BY FILE SIZE

- **read()** loads everything. Fine for small files.
- **for line in f** streams line by line. Best for large files.
- **readlines()** builds a list of all lines.

UNDER THE HOOD

A file object keeps a position marker. Each read continues where the previous one stopped, which is why **rest** begins with the second line.

TRY IT YOURSELF

Write a program that counts the lines, words and characters of **data.txt**.

Paths with pathlib

PYTHON

```python
from pathlib import Path

folder = Path("reports")
folder.mkdir(exist_ok=True)
file = folder / "summary.txt"   # the slash joins paths
file.write_text("Sales rose by 12%\n", encoding="utf8")
print(file.name, file.stem, file.suffix, file.parent)
print(file.exists(), file.is_file(), file.stat().st_size)
print(file.read_text(encoding="utf8").strip())
for path in sorted(folder.glob("*.txt")):
    print("found:", path)
file.unlink()   # delete the file
print(file.exists())
```

OUTPUT

```
summary.txt summary .txt reports
True True 18
Sales rose by 12%
found: reports/summary.txt
False
```

KEY IDEA

A **Path** object represents a location on disk and knows how to inspect, read, write and search it.

WHY NOT PLAIN STRINGS

Windows separates folders with a backslash and other systems with a forward slash. **Path** writes the correct separator for you.

TIP

Path.home() and **Path.cwd()** return your home folder and the current working folder. **rglob()** searches all subfolders.

CSV: rows and columns

PYTHON

```
import csv
from pathlib import Path

rows = [{"name": "Ada", "score": 95}, {"name": "Bob", "score": 72}]
with open("scores.csv", "w", newline="", encoding="utf8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerows(rows)

with open("scores.csv", newline="", encoding="utf8") as f:
    for row in csv.DictReader(f):
        print(row["name"], int(row["score"]) + 1)
print(Path("scores.csv").read_text(encoding="utf8"), end="")
```

OUTPUT

```
Ada 96
Bob 73
name,score
Ada,95
Bob,72
```

VOCABULARY

CSV: comma separated values. One line per record, with fields divided by commas. Every spreadsheet can open it.

KEY IDEA

DictReader turns each row into a dictionary keyed by the header line. Every value arrives as a **string**.

WATCH OUT

Do not split lines on commas by hand. Quoted fields may contain commas, and the **csv** module handles them correctly.

JSON: structured data as text

PYTHON

```python
import json

config = {"name": "demo", "debug": True, "ports": [80, 443], "owner": None}

text = json.dumps(config)   # Python object to JSON text
print(text)
back = json.loads(text)   # JSON text to Python object
print(back["ports"][1], back == config)

with open("config.json", "w", encoding="utf8") as f:
    json.dump(config, f, indent=2)   # straight into a file
with open("config.json", encoding="utf8") as f:
    print(json.load(f)["name"])
```

OUTPUT

```
{"name": "demo", "debug": true, "ports": [80, 443], "owner": null}
443 True
demo
```

HOW THE TYPES MAP

- **dict** becomes a JSON object.
- **list** and **tuple** become an array.
- **True**, **False**, **None** become **true**, **false**, **null**.
- **str**, **int** and **float** keep their meaning.

TIP

Remember the letter **s**: **dumps** and **loads** work with **s**trings, while **dump** and **load** work with files.

WATCH OUT

JSON has no sets, dates or tuples. Convert them to lists or strings first.

Binary files and pickle

PYTHON

```
data = bytes([80, 121, 116, 104, 111, 110])
with open("blob.bin", "wb") as f:   # b means binary
    f.write(data)
with open("blob.bin", "rb") as f:
    raw = f.read()
print(raw, len(raw), raw[0])

import pickle
record = {"id": 7, "tags": {"a", "b"}, "point": (1, 2)}
with open("record.pkl", "wb") as f:
    pickle.dump(record, f)   # almost any object
with open("record.pkl", "rb") as f:
    print(pickle.load(f) == record)
```

OUTPUT

```
b'Python' 6 80
True
```

VOCABULARY

bytes: an immutable sequence of integers from 0 to 255. Images, audio and network packets are all bytes.

PICKLE

pickle saves nearly any Python object, including sets and your own classes, in a format that only Python reads.

WATCH OUT

Never unpickle data from a source you do not trust. Loading a pickle can execute arbitrary code. Prefer JSON for data exchange.

Text encodings: from characters to bytes

PYTHON

```
text = "café €5"
data = text.encode("utf8")   # str to bytes
print(data)
print(len(text), len(data))   # characters versus bytes
print(data.decode("utf8"))   # bytes back to str

print("é".encode("utf8"), "é".encode("latin1"))
print(ord("A"), chr(8364), "\u00e9")
print(data.decode("ascii", errors="ignore"))
```

OUTPUT

```
b'caf\xc3\xa9 \xe2\x82\xac'
7 10
café €5
b'\xc3\xa9' b'\xe9'
65 € é
caf 5
```

VOCABULARY

Encoding: a rule that maps each character to one or more bytes. UTF 8 is the modern standard and covers every language.

KEY IDEA

A **str** holds abstract characters. A file or network holds bytes. **encode()** and **decode()** translate between the two worlds.

UNDER THE HOOD

In UTF 8 the plain English letters take one byte each, the letter é takes two and the euro sign takes three.

Module 8 recap

1. Open files inside a **with** block so that they are always closed.
2. Mode **'r'** reads, **'w'** overwrites, **'a'** appends, **'x'** creates and **'b'** switches to bytes.
3. Loop over a file object to process large files one line at a time.
4. **pathlib.Path** joins, inspects, reads, writes and searches paths on every operating system.
5. Use the **csv** module for tables and the **json** module for nested data.
6. Text is characters, files are bytes. Always state the encoding, and prefer UTF 8.

Coming next: Module 9 shows how to anticipate and handle errors.

09

MODULE 09 INTERMEDIATE

Errors and Exceptions

Programs fail. Good programs fail clearly, recover where they can and always clean up.

IN THIS MODULE

1. The structure of a traceback
2. try, except, else and finally
3. The exception family tree
4. Raising and defining exceptions
5. Two styles: ask or check
6. Context managers

The structure of a traceback

PYTHON

```python
def average(values):
    return sum(values) / len(values)

def report(data):
    print("Average:", average(data))

report([])
```

3 2 1

TRACEBACK

```
Traceback (most recent call last):
  File "demo.py", line 7, in <module>
    report([])
  File "demo.py", line 5, in report
    print("Average:", average(data))
  File "demo.py", line 2, in average
    return sum(values) / len(values)
ZeroDivisionError: division by zero
```

READ IT IN THIS ORDER

- The **last line** names the exception type and the reason.
- The frames above it show the chain of calls, oldest first.
- The frame closest to the bottom is where the error was raised.

KEY IDEA

The yellow tags follow the calls: the main program called **report**, which called **average**, which failed.

TIP

The bug is not always in the last frame. Here **average** is fine. The real mistake is passing an empty list.

Catching errors with try and except

PYTHON

```python
def to_number(text):
    try:
        return int(text)
    except ValueError:
        print(f"{text!r} is not a whole number")
        return None

print(to_number("42"))
print(to_number("4x"))

try:
    result = 10 / 0
except (ZeroDivisionError, TypeError) as error:
    print("Problem:", error, "|", type(error).__name__)
```

OUTPUT

```
42
'4x' is not a whole number
None
Problem: division by zero | ZeroDivisionError
```

IN PLAIN ENGLISH

Try to convert the text. If that raises a **ValueError**, *explain the problem and give back* **None**.

KEY IDEA

When a line inside **try** raises, Python skips the rest of the block and jumps to the first matching **except**.

DETAILS

- A tuple of types catches several exceptions at once.
- **as error** binds the exception object so that you can inspect it.

The full statement: else and finally

PYTHON

```
def read_ratio(a, b):
    try:
        ratio = a / b   # the risky step
    except ZeroDivisionError:
        print("  except: cannot divide by zero")
        return None
    else:
        print("  else: no error happened")
        return ratio
    finally:
        print("  finally: always runs")

print(read_ratio(6, 3))
print(read_ratio(6, 0))
```

OUTPUT

```
  else: no error happened
  finally: always runs
2.0
  except: cannot divide by zero
  finally: always runs
None
```

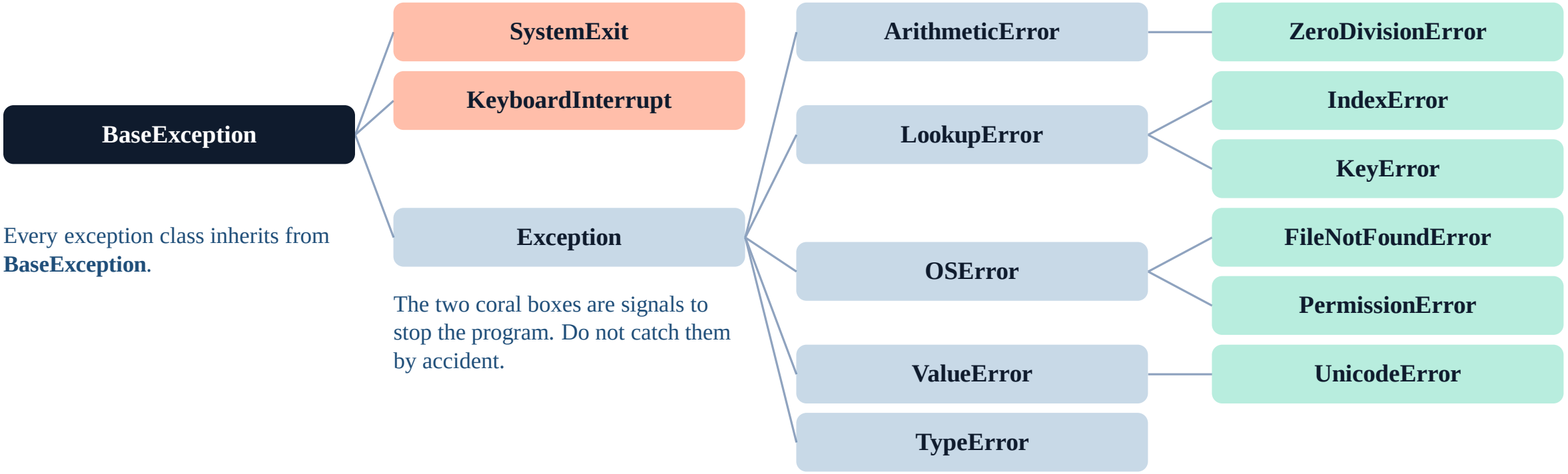
FOUR CLAUSES

- **try**: the code that might fail.
- **except**: runs only when a matching error occurred.
- **else**: runs only when **no** error occurred.
- **finally**: runs in every case, even after a **return**.

TIP

Put cleanup work, such as closing a connection, in **finally**.

The exception family tree



Every exception class inherits from **BaseException**.

The two coral boxes are signals to stop the program. Do not catch them by accident.

KEY IDEA

except Exception catches every ordinary error and still lets Ctrl C and **sys.exit()** through.

TIP

Catching a parent catches its children: **except LookupError** handles both **IndexError** and **KeyError**.

WATCH OUT

A bare **except:** swallows everything, including typing mistakes in your own code. Name the type.

Raising and defining exceptions

PYTHON

```
class InsufficientFunds(Exception):
    """Raised when a withdrawal exceeds the balance."""
    def __init__(self, balance, amount):
        super().__init__(f"need {amount}, have {balance}")
        self.shortfall = amount - balance

def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("amount must be positive")
    if amount > balance:
        raise InsufficientFunds(balance, amount)
    return balance - amount

try:
    withdraw(50, 80)
except InsufficientFunds as error:
    print(error, "| short by", error.shortfall)
```

OUTPUT

```
need 80, have 50 | short by 30
```

KEY IDEA

raise creates an exception and stops normal flow. Use it the moment a function receives input it cannot honour.

YOUR OWN EXCEPTION TYPES

Inherit from **Exception**. A specific class lets callers catch exactly your error and carry extra data, such as the shortfall here.

TIP

Raise the most specific built in type that fits: **ValueError** for a bad value, **TypeError** for a wrong type. Module 10 explains classes in full.

Two styles: check first or try first

LOOK BEFORE YOU LEAP

```
config = {"port": "8080"}

if "port" in config and config["port"].isdigit():
    port = int(config["port"])
else:
    port = 80
print(port)
```

OUTPUT

```
8080
```

ASK FORGIVENESS

```
config = {"port": "8080"}

try:
    port = int(config["port"])
except (KeyError, ValueError):
    port = 80
print(port)
```

OUTPUT

```
8080
```

VOCABULARY

EAFP: it is easier to ask forgiveness than permission. Attempt the operation and handle the failure.

KEY IDEA

Python favours the **try** style. The happy path stays readable, and no gap exists between the check and the action.

WATCH OUT

Keep the **try** block small and catch **specific** types, or you will hide unrelated bugs.

Chaining and passing exceptions on

raise ... from

```
def load_port(settings):
    try:
        return int(settings["port"])
    except KeyError as error:
        raise RuntimeError("incomplete config") from error
try:
    load_port({})
except RuntimeError as error:
    print("Caught:", error)
    print("Cause:", repr(error.__cause__))
```

OUTPUT

```
Caught: incomplete config
Cause: KeyError('port')
```

TRANSLATE

raise New(...) from error turns a low level failure into a meaningful one and keeps the original as **__cause__**.

A BARE raise

```
def careful():
    try:
        return 1 / 0
    except ZeroDivisionError:
        print("log the problem, then pass it on")
        raise   # the same exception continues
try:
    careful()
except ZeroDivisionError:
    print("handled at the top level")
```

OUTPUT

```
log the problem, then pass it on
handled at the top level
```

PASS ON

A bare **raise** inside **except** sends the current exception onward unchanged.

Context managers and the with statement

PYTHON

```
class Timer:
    def __enter__(self):
        print("enter: start the clock")
        return self
    def __exit__(self, exc_type, exc, traceback):
        print("exit: stop the clock, even after an error")
        return False   # do not swallow exceptions

with Timer() as t:
    print("working inside the block")

with open("a.txt", "w") as src, open("b.txt", "w") as dst:
    src.write("x")   # both files close on exit
print(src.closed, dst.closed)
```

OUTPUT

```
enter: start the clock
working inside the block
exit: stop the clock, even after an error
True True
```

VOCABULARY

Context manager: an object with **__enter__** and **__exit__** methods that sets something up and reliably tears it down.

KEY IDEA

with guarantees that **__exit__** runs when the block ends, whether it ends normally, by **return** or by an exception.

TIP

Files, locks, database transactions and temporary folders are all managed this way. Module 12 shows a shorter way to write your own.

Common exceptions and what they tell you

Exception	Typical cause	One line that triggers it
SyntaxError	The code breaks the grammar of the language	**if x > 3 print(x)**
NameError	A name was never defined, often a typing mistake	**print(totl)**
TypeError	An operation received a value of the wrong type	**'2' + 2**
ValueError	The type is right but the value is unacceptable	**int('two')**
IndexError	A sequence position does not exist	**[1, 2][5]**
KeyError	A dictionary key does not exist	**{'a': 1}['b']**
AttributeError	The object has no such attribute or method	**None.upper()**
FileNotFoundError	The path does not lead to a file	**open('nope.txt')**

TIP

Search for the exception name together with the last line of the message. Someone has almost certainly met the same problem.

TRY IT YOURSELF

Trigger each of these eight errors on purpose in the shell and read every traceback.

Quiz: return and finally

PREDICT THE OUTPUT

```
def f():
    try:
        return "try"
    finally:
        print("finally")

print(f())
```

ACTUAL OUTPUT

```
finally
try
```

QUESTION

In which order do the two words appear?

OPTIONS

A. Only **try**

B. **finally** and then **try**

C. **try** and then **finally**

D. Only **finally**

ANSWER

B. The **finally** block runs after the return value is prepared but before the function actually hands it back.

Module 9 recap

1. Read a traceback from the bottom: exception type and message first, then the chain of calls.
2. **try** guards risky code. **except** handles named exception types and can bind the error with **as**.
3. **else** runs when nothing failed. **finally** always runs and is the home of cleanup code.
4. Exceptions form a class tree. Catching a parent also catches every child.
5. **raise** signals a problem. Define your own exception classes by inheriting from **Exception**.
6. Python style prefers trying over checking, with small **try** blocks and specific types.
7. **raise ... from** translates an error. A bare **raise** passes it on.
8. **with** and context managers guarantee that resources are released.

Coming next: Module 10 builds your own data types with classes.

10

MODULE 10 INTERMEDIATE

Object Oriented Programming

Bundle data and the behaviour that belongs to it into classes, then build families of related types.

IN THIS MODULE

1. Classes, instances and self
2. Attributes, methods and properties
3. Inheritance and super
4. Polymorphism and duck typing
5. Special methods and protocols
6. Dataclasses, enums and slots
7. Composition

Classes and objects: blueprint and buildings

CLASS: THE BLUEPRINT

class Dog

Data every dog has
name, **age**

Behaviour every dog has
bark(), **birthday()**

Dog('Rex', 3)
creates an instance

INSTANCES: THE BUILDINGS

rex	name = 'Rex', age = 3
luna	name = 'Luna', age = 5
milo	name = 'Milo', age = 1

VOCABULARY

Class: a template that describes what its objects know and what they can do.

VOCABULARY

Instance: one concrete object built from a class, with its own data.

VOCABULARY

Attribute: a piece of data stored on an object, reached with a dot.

VOCABULARY

Method: a function defined in a class that acts on one instance.

KEY IDEA

One class, many instances. Each instance shares the behaviour and carries its own values.

Defining a class: __init__ and self

PYTHON

```python
class Dog:
    """A dog with a name and an age."""
    def __init__(self, name, age):   # runs for every new Dog
        self.name = name   # data stored on the instance
        self.age = age

    def bark(self):
        return f"{self.name} says woof!"

    def birthday(self):
        self.age += 1

rex = Dog("Rex", 3)   # Python calls __init__
luna = Dog("Luna", 5)
rex.birthday()
print(rex.bark(), rex.age, luna.age)
print(type(rex).__name__, isinstance(rex, Dog))
```

OUTPUT

```
Rex says woof! 4 5
Dog True
```

LINE BY LINE

1. **__init__** is the initialiser. Python calls it automatically for each new object.
2. **self** is the object being built. Whatever you attach to it belongs to that one instance.
3. Every method receives the instance as its first parameter, named **self** by convention.
4. Calling the class creates an instance. You never pass **self** yourself.

UNDER THE HOOD

rex.bark() is shorthand for **Dog.bark(rex)**. The object before the dot becomes **self**.

Instance attributes and class attributes

PYTHON

```python
class Dog:
    species = "Canis familiaris"   # class attribute: shared
    count = 0

    def __init__(self, name):
        self.name = name   # instance attribute: own copy
        Dog.count += 1

a, b = Dog("Rex"), Dog("Luna")
print(a.species, "|", b.species, "|", Dog.count)
a.species = "Robot dog"   # creates an instance attribute
print(a.species, "|", b.species)
print(a.__dict__, b.__dict__)
```

OUTPUT

```
Canis familiaris | Canis familiaris | 2
Robot dog | Canis familiaris
{'name': 'Rex', 'species': 'Robot dog'} {'name': 'Luna'}
```

KEY IDEA

Instance attributes live on one object. Class attributes live on the class and are visible through every instance.

UNDER THE HOOD

Attribute lookup checks the instance first and the class second. Assigning through an instance always writes to the instance and hides the class value.

WATCH OUT

A mutable class attribute, such as a list, is shared by **all** instances. Create per object containers inside **__init__**.

Instance, class and static methods

PYTHON

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    def to_fahrenheit(self):   # instance method: self
        return self.celsius * 9 / 5 + 32

    @classmethod
    def from_fahrenheit(cls, f):   # class method: cls
        return cls((f - 32) * 5 / 9)

    @staticmethod
    def is_valid(celsius):   # static method: neither
        return celsius >= -273.15

t = Temperature.from_fahrenheit(212)
print(t.celsius, t.to_fahrenheit(), Temperature.is_valid(-300))
```

OUTPUT

```
100.0 212.0 False
```

THREE KINDS OF METHOD

- An **instance method** works on one object through **self**.
- A **class method** receives the class as **cls**. It is ideal for alternative constructors.
- A **static method** receives nothing extra. It is a plain function kept in the class for tidiness.

TIP

Using **cls(...)** in place of the class name makes the alternative constructor work correctly for subclasses too.

Readable objects: __repr__ and __str__

PYTHON

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):   # for developers
        return f"Point(x={self.x}, y={self.y})"

    def __str__(self):   # for end users
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)   # uses __str__
print(repr(p))   # uses __repr__
print([p, Point(0, 0)])   # containers show the repr
print(f"{p} and {p!r}")
```

OUTPUT

```
(3, 4)
Point(x=3, y=4)
[Point(x=3, y=4), Point(x=0, y=0)]
(3, 4) and Point(x=3, y=4)
```

KEY IDEA

__repr__ should be unambiguous and, ideally, look like the code that would recreate the object. **__str__** should be pleasant to read.

WITHOUT THEM

Python prints only the class name and a memory address, which tells you nothing about the content.

TIP

If you write only one of the two, write **__repr__**. Python falls back to it whenever **__str__** is missing.

Properties: attributes with rules

PYTHON

```python
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance   # underscore: internal
    @property
    def balance(self):   # read like an attribute
        return self._balance
    @balance.setter
    def balance(self, value):   # checked on assignment
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value

acct = Account("Ada", 100)
acct.balance += 50   # getter, then setter
print(acct.balance)
acct.balance = -1
```

OUTPUT

```
150
ValueError: balance cannot be negative
```

VOCABULARY

Property: a method that is used like an attribute. The caller writes **acct.balance**, with no parentheses.

KEY IDEA

Start with plain attributes. Add a property later, when you need validation or a computed value, and no calling code has to change.

TIP

A property without a setter is read only. Any attempt to assign to it raises **AttributeError**.

Privacy by convention

PYTHON

```
class Vault:
    def __init__(self):
        self.label = "public"   # anyone may use it
        self._hint = "internal"   # please do not touch
        self.__code = 1234   # name mangled by Python

v = Vault()
print(v.label, v._hint)
print(v._Vault__code)   # still reachable
print(list(vars(v)))
print(hasattr(v, "__code"))
```

OUTPUT

```
public internal
1234
['label', '_hint', '_Vault__code']
False
```

THREE LEVELS

- No underscore: part of the public interface.
- One leading underscore: internal detail, used at your own risk.
- Two leading underscores: Python renames the attribute to avoid clashes in subclasses.

KEY IDEA

Python has no truly private attributes. The underscore is a polite sign on the door, and programmers respect it.

IN PLAIN ENGLISH

We are all consenting adults here: the language trusts you not to reach into the internals of an object.

Inheritance: building on an existing class

PYTHON

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
    def describe(self):
        return f"{self.name} says {self.speak()}"

class Dog(Animal):   # Dog inherits from Animal
    def speak(self):   # override one method
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

for pet in [Dog("Rex"), Cat("Tom"), Animal("Blob")]:
    print(pet.describe())
print(isinstance(Dog("x"), Animal), issubclass(Cat, Animal))
```

OUTPUT

```
Rex says Woof
Tom says Meow
Blob says ...
True True
```

VOCABULARY

Inheritance: a child class receives every attribute and method of its parent and may add or replace some.

KEY IDEA

describe is written once in **Animal**, yet it calls the **speak** of whichever class the object really is.

Extending the parent with super()

PYTHON

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def summary(self):
        return f"{self.name} earns {self.salary}"

class Manager(Employee):
    def __init__(self, name, salary, reports):
        super().__init__(name, salary)   # reuse the parent setup
        self.reports = reports
    def summary(self):
        base = super().summary()   # extend, not replace
        return f"{base} and leads {len(self.reports)} people"

boss = Manager("Grace", 9000, ["Ada", "Alan"])
print(boss.summary())
```

OUTPUT

Grace earns 9000 and leads 2 people

KEY IDEA

super() gives access to the parent implementation, so the child can add to it without copying it.

WATCH OUT

If a child defines **__init__** and forgets **super().__init__(...)**, the attributes set by the parent never come into existence.

TRY IT YOURSELF

Add a class **Intern(Employee)** whose summary ends with the words **(intern)**. Reuse the parent method.

Polymorphism and duck typing

PYTHON

```
class Duck:
    def speak(self):
        return "Quack"

class Robot:   # no shared parent class
    def speak(self):
        return "Beep"

def announce(thing):
    print(thing.speak())   # anything that can speak

for item in [Duck(), Robot()]:
    announce(item)
print(len("abc"), len([1, 2]), len({"a": 1}))
announce(42)
```

OUTPUT

```
Quack
Beep
3 2 1
AttributeError: 'int' object has no attribute 'speak'
```

VOCABULARY

Polymorphism: one piece of code works with objects of many types, as long as each type supports the operation.

VOCABULARY

Duck typing: if it walks like a duck and quacks like a duck, treat it as a duck. Python checks what an object can **do**, not what it **is**.

TIP

Avoid **isinstance** checks before calling a method. Call it, and let unsuitable objects fail loudly.

Multiple inheritance and the MRO

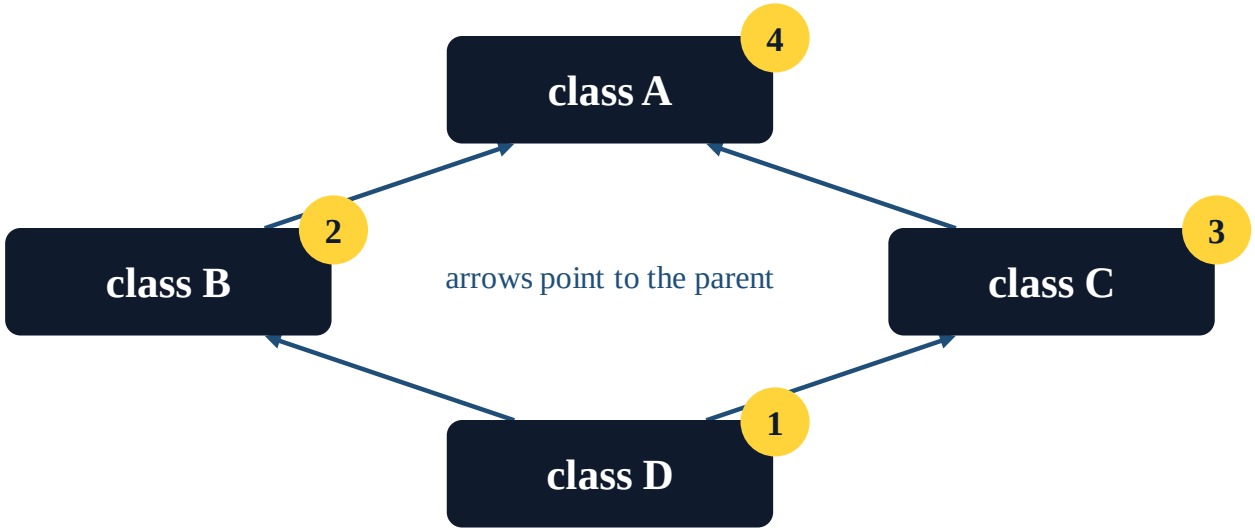
PYTHON

```
class A:
    def hello(self):
        return "A"
class B(A):
    def hello(self):
        return "B > " + super().hello()
class C(A):
    def hello(self):
        return "C > " + super().hello()
class D(B, C):
    pass

print(D().hello())
print([cls.__name__ for cls in D.__mro__])
```

OUTPUT

```
B > C > A
['D', 'B', 'C', 'A', 'object']
```



VOCABULARY

MRO: the method resolution order. It is the single line of classes, shown by the yellow numbers, that Python searches for a method.

KEY IDEA

super() means the **next class in the MRO**, not simply the parent. That is why B hands over to C before A is reached.

Abstract base classes

PYTHON

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        """Every concrete shape must implement this."""

    def describe(self):
        return f"{type(self).__name__} with area {self.area():.2f}"

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

print(Circle(1).describe())
Shape()   # refused by Python
```

OUTPUT

```
Circle with area 3.14
TypeError: Can't instantiate abstract class Shape without an implementation for abstract method 'area'
```

VOCABULARY

Abstract base class: a class that defines an interface and cannot be instantiated itself.

KEY IDEA

A subclass must implement every method marked **@abstractmethod** before Python allows any instance to be created.

Operator overloading with special methods

PYTHON

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
    def __add__(self, other):   # v + w
        return Vector(self.x + other.x, self.y + other.y)
    def __mul__(self, k):   # v * 3
        return Vector(self.x * k, self.y * k)
    def __rmul__(self, k):   # 3 * v
        return self * k
    def __eq__(self, other):   # v == w
        return (self.x, self.y) == (other.x, other.y)
    def __abs__(self):   # abs(v)
        return (self.x ** 2 + self.y ** 2) ** 0.5

v, w = Vector(3, 4), Vector(1, 2)
print(v + w, v * 2, 2 * v, v == Vector(3, 4), abs(v))
```

OUTPUT

```
Vector(4, 6) Vector(6, 8) Vector(6, 8) True 5.0
```

VOCABULARY

Special method: a method whose name starts and ends with two underscores. Python calls it when an operator or built in function is applied.

KEY IDEA

v + w is translated into **v.__add__(w)**. Define the method and your own class supports the operator.

TIP

Overload an operator only when its meaning is obvious. Adding two vectors is clear. Adding two customers is not.

Making a class behave like a container

PYTHON

```
class Playlist:
    def __init__(self, *songs):
        self._songs = list(songs)
    def __len__(self):   # len(p)
        return len(self._songs)
    def __getitem__(self, index):   # p[0], slices, loops
        return self._songs[index]
    def __contains__(self, song):   # "x" in p
        return song in self._songs

p = Playlist("Intro", "Verse", "Outro")
print(len(p), p[0], p[-1], p[0:2])
print("Verse" in p, bool(p))
print([song.upper() for song in p])
```

OUTPUT

```
3 Intro Outro ['Intro', 'Verse']
True True
['INTRO', 'VERSE', 'OUTRO']
```

VOCABULARY

Protocol: a small set of special methods that Python looks for. Any class that supplies them gains the matching behaviour.

KEY IDEA

With only **__len__** and **__getitem__** the playlist supports indexing, slicing, **for** loops, **in**, **sorted()** and truth testing.

UNDER THE HOOD

bool(p) uses **__len__** when no **__bool__** exists. An empty playlist is therefore falsy, exactly like an empty list.

Equality and hashing

PYTHON

```
class Card:
    def __init__(self, rank, suit):
        self.rank, self.suit = rank, suit
    def __eq__(self, other):
        if not isinstance(other, Card):
            return NotImplemented
        return (self.rank, self.suit) == (other.rank, other.suit)
    def __hash__(self):   # equal objects, equal hash
        return hash((self.rank, self.suit))

a, b = Card("A", "spades"), Card("A", "spades")
print(a == b, a is b)
print(len({a, b}), {a: "ace"}[b])
```

OUTPUT

```
True False
1 ace
```

KEY IDEA

By default two objects are equal only when they are the same object. Define **__eq__** to compare by content.

THE HASH RULE

Objects that compare equal must have the same hash. A class that defines **__eq__** without **__hash__** becomes unhashable and cannot enter a set or act as a dictionary key.

TIP

Return **NotImplemented** for foreign types. Python then gives the other object a chance to answer.

Dataclasses: classes without the boilerplate

PYTHON

```python
from dataclasses import dataclass, field

@dataclass
class Product:
    name: str
    price: float
    tags: list[str] = field(default_factory=list)

    def with_tax(self, rate=0.2):
        return round(self.price * (1 + rate), 2)

p = Product("Tea", 2.5)
q = Product("Tea", 2.5)
print(p)   # __repr__ written for you
print(p == q, p.with_tax())   # __eq__ written for you
p.tags.append("drink")
print(p.tags, q.tags)   # each object has its own list
```

OUTPUT

```
Product(name='Tea', price=2.5, tags=[])
True 3.0
['drink'] []
```

KEY IDEA

@dataclass reads the annotated fields and writes **__init__**, **__repr__** and **__eq__** for you.

MUTABLE DEFAULTS

field(default_factory=list) builds a fresh list for every instance. A plain **= []** is rejected, which protects you from the shared default trap.

TIP

Reach for a dataclass whenever a class mainly stores data.

Frozen dataclasses and enumerations

PYTHON

```python
from dataclasses import dataclass
from enum import Enum
@dataclass(frozen=True, order=True)
class Version:
    major: int
    minor: int
print(Version(3, 12) > Version(3, 9))
print(sorted([Version(2, 0), Version(1, 5)])[0])

class Color(Enum):
    RED = 1
    GREEN = 2
print(Color.RED, Color.RED.name, Color.RED.value)
print(Color(2), Color["GREEN"] is Color.GREEN, len(Color))
```

OUTPUT

```
True
Version(major=1, minor=5)
Color.RED RED 1
Color.GREEN True 2
```

DATACLASS OPTIONS

- **frozen=True** makes instances immutable and hashable.
- **order=True** adds comparison operators that compare field by field.
- **slots=True** saves memory.

VOCABULARY

Enum: a fixed set of named constants. **Color.RED** is safer and clearer than a bare number or string.

TIP

Use an enum for any value with a closed list of choices: status, direction, weekday, log level.

Saving memory with __slots__

PYTHON

```python
class PointDict:
    def __init__(self, x, y):
        self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")   # a fixed set of attributes
    def __init__(self, x, y):
        self.x, self.y = x, y

a, b = PointDict(1, 2), PointSlots(1, 2)
a.z = 3   # fine: stored in a.__dict__
print(a.__dict__, hasattr(b, "__dict__"))
import sys
print(sys.getsizeof(a) + sys.getsizeof(a.__dict__) > sys.getsizeof(b))
b.z = 3   # no room for a new attribute
```

OUTPUT

```
{'x': 1, 'y': 2, 'z': 3} False
True
AttributeError: 'PointSlots' object has no attribute 'z'
```

UNDER THE HOOD

A normal instance stores its attributes in a private dictionary, **__dict__**. That is flexible but costs memory.

KEY IDEA

__slots__ replaces the dictionary with fixed storage. Objects become smaller and attribute access a little faster.

TIP

Slots pay off when a program creates hundreds of thousands of small objects. Otherwise leave them out.

Composition: has a, not is a

PYTHON

```python
class Engine:
    def __init__(self, power):
        self.power = power
    def start(self):
        return f"engine with {self.power} kW started"

class Car:   # a Car HAS an Engine
    def __init__(self, model, engine):
        self.model = model
        self.engine = engine    # composition
    def start(self):
        return f"{self.model}: {self.engine.start()}"

print(Car("Roadster", Engine(215)).start())
print(Car("City", Engine(60)).start())
```

OUTPUT

```
Roadster: engine with 215 kW started
City: engine with 60 kW started
```

TWO RELATIONSHIPS

- **Is a**: a Dog is an Animal. Use inheritance.
- **Has a**: a Car has an Engine. Use composition.

KEY IDEA

Composition builds complex objects out of simple parts. Parts can be swapped, tested alone and reused elsewhere.

TIP

Prefer composition. Deep inheritance trees are rigid, and a change near the root ripples through every descendant.

Quiz: class and instance attributes

PREDICT THE OUTPUT

```
class Counter:
    total = 0
    def __init__(self):
        self.total += 1

a = Counter()
b = Counter()
print(Counter.total, a.total, b.total)
```

ACTUAL OUTPUT

```
0 1 1
```

QUESTION

What is printed?

OPTIONS

A. 2 2 2

B. 0 1 1

C. 2 1 1

D. 0 0 0

ANSWER

B. `self.total += 1` reads the class value 0 and then stores 1 on the **instance**. The class attribute never changes.

Module 10 recap

1. A class is a blueprint. Calling it creates an instance, and **__init__** sets that instance up.
2. **self** is the instance. Class attributes are shared, instance attributes are individual.
3. **@property** adds rules to attribute access. **@classmethod** builds alternative constructors.
4. A child class inherits everything, overrides what differs and reaches the parent through **super()**.
5. Duck typing: code depends on what an object can do. The MRO decides where a method is found.
6. Special methods connect your class to operators, **len()**, indexing, printing and hashing.
7. **@dataclass** writes the routine methods. **Enum** names a closed set of constants.
8. Favour composition for has a relationships and keep inheritance trees shallow.

Coming next: Module 11 looks inside the for loop: iterators and generators.

11

MODULE 11 ADVANCED

Iterators and Generators

Look inside the for loop, then produce values one at a time, on demand, with almost no memory.

IN THIS MODULE

1. The iteration protocol
2. Writing an iterator class
3. Generator functions and yield
4. Generator expressions and laziness
5. The itertools module
6. Pipelines, yield from and send

The iteration protocol

PYTHON

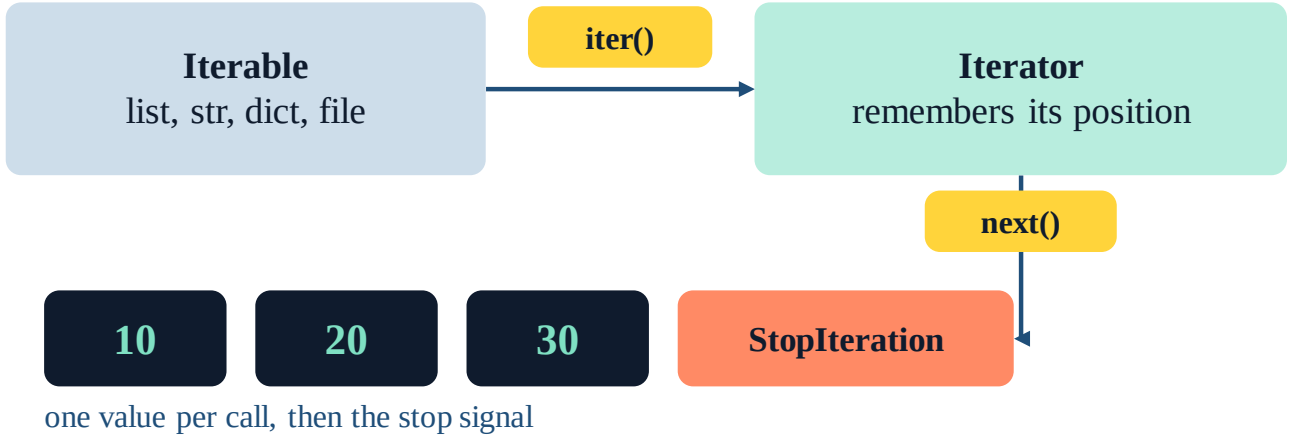
```
numbers = [10, 20, 30]
it = iter(numbers)   # ask the list for an iterator
print(next(it))
print(next(it))
print(next(it))
print(next(it, "done"))   # a default instead of an error
```

OUTPUT

```
10
20
30
done
```

VOCABULARY

Iterable: an object that can hand out an iterator. **Iterator**: an object that produces the next value on request.



one value per call, then the stop signal

WHAT A FOR LOOP REALLY DOES

```
it = iter(numbers)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
    print(item)
```

KEY IDEA

A **for** loop calls **iter()** once, then **next()** again and again until **StopIteration** appears.

Writing an iterator class

PYTHON

```
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):   # an iterator returns itself
        return self
    def __next__(self):   # produce the next value
        if self.current <= 0:
            raise StopIteration   # signal: nothing is left
        value = self.current
        self.current -= 1
        return value

for n in Countdown(3):
    print(n, end=" ")
print()
c = Countdown(2)
print(list(c), list(c))   # an iterator is used up once
```

OUTPUT

```
3 2 1
[2, 1] []
```

THE TWO METHODS

- **__iter__** returns the iterator object.
- **__next__** returns the next value or raises **StopIteration**.

KEY IDEA

Any class with these two methods works in a **for** loop, in **list()**, in **sum()** and everywhere else Python iterates.

WATCH OUT

An iterator is a one way street. Once exhausted it stays empty, which is why the second **list(c)** is empty.

Generator functions: yield

PYTHON

```
def countdown(start):
    while start > 0:
        yield start   # hand out a value and pause here
        start -= 1

for n in countdown(3):
    print(n, end=" ")
print()

gen = countdown(2)
print(type(gen).__name__, next(gen), next(gen))
print(list(countdown(4)), sum(countdown(4)))
```

OUTPUT

```
3 2 1
generator 2 1
[4, 3, 2, 1] 10
```

VOCABULARY

Generator function: a function that contains **yield**. Calling it returns a generator, which is an iterator written for you.

KEY IDEA

These four lines do the same job as the eleven line **Countdown** class on the previous slide.

IN PLAIN ENGLISH

Each time the loop asks for a value, the function runs until the next ***yield****, hands over the value and freezes.*

How yield pauses and resumes a function

PYTHON

```
def chatty():
    print("  started")
    yield 1
    print("  resumed after the first yield")
    yield 2
    print("  resumed again, now finishing")
gen = chatty()
print("generator created, nothing has run yet")
print("got", next(gen))
print("got", next(gen))
print("got", next(gen, "nothing more"))
```

1 2 3 4

OUTPUT

```
generator created, nothing has run yet
  started
got 1
  resumed after the first yield
got 2
  resumed again, now finishing
got nothing more
```

FOLLOW THE TAGS

1. Calling **chatty()** runs **no** code. It only builds the generator.
2. The first **next()** runs up to the first **yield**.
3. The second **next()** continues from that exact spot.
4. The third finds no further **yield**, so the generator ends.

UNDER THE HOOD

Between calls the generator keeps its local variables and its position alive in a suspended frame.

Generator expressions and lazy evaluation

PYTHON

```
import sys
squares_list = [n * n for n in range(1_000_000)]
squares_gen = (n * n for n in range(1_000_000))
print(sys.getsizeof(squares_list) > 8_000_000)
print(sys.getsizeof(squares_gen) < 500)

print(sum(n * n for n in range(10)))   # no extra brackets
print(any(len(w) > 5 for w in ["tea", "coffee"]))
first = next(n for n in range(1, 100) if n % 17 == 0)
print(first)
lines = (line.strip() for line in ["  a ", " b  "])
print(list(lines), list(lines))   # used up after one pass
```

OUTPUT

```
True
True
285
True
17
['a', 'b'] []
```

VOCABULARY

Lazy evaluation: a value is computed only at the moment someone asks for it.

KEY IDEA

The list holds a million results in memory. The generator holds only the recipe and produces one result at a time.

TIP

Feed a generator expression straight into **sum()**, **max()**, **any()**, **all()** or **''.join()**.

The itertools toolbox

PYTHON

```
from itertools import count, cycle, islice, chain
from itertools import pairwise, accumulate, combinations, groupby

print(list(islice(count(10, 5), 4)))   # endless, then sliced
print(list(islice(cycle("AB"), 5)))
print(list(chain([1, 2], "ab")))
print(list(pairwise([1, 2, 3, 4])))
print(list(accumulate([1, 2, 3, 4])))
print(list(combinations("ABC", 2)))
print([(k, len(list(g))) for k, g in groupby("aaabbc")])
```

OUTPUT

```
[10, 15, 20, 25]
['A', 'B', 'A', 'B', 'A']
[1, 2, 'a', 'b']
[(1, 2), (2, 3), (3, 4)]
[1, 3, 6, 10]
[('A', 'B'), ('A', 'C'), ('B', 'C')]
[('a', 3), ('b', 2), ('c', 1)]
```

WHAT EACH ONE DOES

- **count** and **cycle** never end.
- **islice** cuts a piece out of any iterator.
- **chain** joins iterables end to end.
- **pairwise** yields neighbours.
- **accumulate** yields running totals.
- **groupby** groups equal neighbours.

WATCH OUT

Never call **list()** on an endless iterator. Slice it first.

Generator pipelines

PYTHON

```
def read_lines(rows):
    for row in rows:
        yield row

def parse(lines):
    for line in lines:
        name, amount = line.split(",")
        yield name, float(amount)

def only_large(records, limit):
    return (r for r in records if r[1] >= limit)

raw = ["tea,2.5", "laptop,950", "pen,1.2", "phone,600"]
pipeline = only_large(parse(read_lines(raw)), 100)
for name, amount in pipeline:
    print(name, amount)
```

OUTPUT

```
laptop 950.0
phone 600.0
```

KEY IDEA

Each stage pulls one item from the stage before it. No stage ever holds the whole data set.

IN PLAIN ENGLISH

The loop asks the filter for a record, the filter asks the parser, and the parser asks the reader for a single line.

TIP

Replace **raw** with an open file of ten million lines and the program still runs in constant memory.

Advanced generators: yield from and send

yield from

```
def inner():
    yield 1
    yield 2
    return "inner done"

def outer():
    result = yield from inner()   # delegate
    print("  inner returned:", result)
    yield 3
print(list(outer()))
```

OUTPUT

```
  inner returned: inner done
[1, 2, 3]
```

DELEGATION

yield from passes every value of another generator through and captures its return value.

send()

```
def running_average():
    total, n, avg = 0, 0, None
    while True:
        value = yield avg   # receive from send()
        total, n = total + value, n + 1
        avg = total / n
ra = running_average()
next(ra)   # move to the first yield
print(ra.send(10), ra.send(20), ra.send(60))
```

OUTPUT

```
10.0 15.0 30.0
```

TWO WAY TRAFFIC

send(x) resumes the generator and makes the paused **yield** evaluate to **x**. This idea led to **async** and **await**.

Quiz: using a generator twice

PREDICT THE OUTPUT

```
gen = (x * 2 for x in range(3))
print(list(gen))
print(list(gen))
```

ACTUAL OUTPUT

```
[0, 2, 4]
[]
```

QUESTION

What do the two lines print?

OPTIONS

A. **[0, 2, 4]** twice

B. **[0, 2, 4]** and then **[]**

C. **[]** twice

D. A **StopIteration** error

ANSWER

B. The first **list()** consumes every value. A generator cannot be rewound, so the second finds nothing.

Module 11 recap

1. An iterable hands out an iterator through **iter()**. An iterator hands out values through **next()**.
2. A **for** loop is **iter()** followed by repeated **next()** until **StopIteration**.
3. A function with **yield** is a generator: it pauses at each **yield** and resumes on the next request.
4. Generator expressions are lazy. They compute one item at a time and use almost no memory.
5. **itertools** offers endless counters, slicing, chaining, grouping and combinations.
6. Pipelines of generators process huge data streams. An iterator can be consumed only once.

Coming next: Module 12 wraps functions with decorators.

12

MODULE 12 ADVANCED

Decorators and Functional Tools

Add behaviour to functions without touching their code, and meet the helpers in functools and contextlib.

IN THIS MODULE

1. Wrapping a function by hand
2. The at sign syntax
3. The structure of a decorator
4. Decorators with arguments
5. Stacking and class based decorators
6. functools and contextlib

Wrapping one function inside another

PYTHON

```python
def shout(func):                                          # 1
    def wrapper(name):                                    # 2
        result = func(name)   # call the original
        return result.upper() + "!"   # then improve the result
    return wrapper                                        # 3

def greet(name):
    return f"hello {name}"

loud_greet = shout(greet)   # build a wrapped version     # 4
print(greet("ada"))
print(loud_greet("ada"))
```

OUTPUT

```
hello ada
HELLO ADA!
```

LINE BY LINE

1. **shout** takes a **function** as its argument.
2. Inside, it defines a new function that calls the original and adds something.
3. It returns that new function without calling it.
4. The result is a second function that behaves like the first, only louder.

KEY IDEA

This works because functions are objects and closures remember **func**. Both ideas come from Module 6.

The at sign is only a shortcut

BY HAND

```
def shout(func):
    def wrapper(name):
        return func(name).upper() + "!"
    return wrapper

def greet(name):
    return f"hello {name}"
greet = shout(greet)   # replace the original

print(greet("ada"))
```

OUTPUT

```
HELLO ADA!
```

WITH A DECORATOR

```
def shout(func):
    def wrapper(name):
        return func(name).upper() + "!"
    return wrapper

@shout   # the same replacement
def greet(name):
    return f"hello {name}"

print(greet("ada"))
```

OUTPUT

```
HELLO ADA!
```

VOCABULARY

Decorator: a callable that takes a function and returns a replacement for it.

KEY IDEA

Writing **@shout** above a definition means exactly **greet = shout(greet)**. Nothing more happens.

UNDER THE HOOD

The decorator runs **once**, at the moment the function is defined, not each time it is called.

The structure of a decorator

PYTHON

```python
import functools

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__}{args}")
        result = func(*args, **kwargs)
        print(f"finished with {result}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b
print(add(2, 3))
```

1 2 3 4 5 6

OUTPUT

```
calling add(2, 3)
finished with 5
5
```

SIX PARTS

1. The decorator receives the original function.
2. The wrapper accepts **any** arguments.
3. Work done **before** the call.
4. The original call, with all arguments passed along.
5. Work done **after** the call.
6. The decorator returns the wrapper.

TIP

Copy this skeleton whenever you need a decorator. Only parts 3 and 5 change.

Keeping the identity of the function

WITHOUT functools.wraps

```
def timer(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@timer
def load():
    """Load the data."""

print(load.__name__, load.__doc__)
```

OUTPUT

```
wrapper None
```

WITH functools.wraps

```
import functools
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@timer
def load():
    """Load the data."""

print(load.__name__, load.__doc__)
```

OUTPUT

```
load Load the data.
```

WATCH OUT

A plain wrapper hides the name and the docstring of the original. Debuggers, logs and **help()** then show the wrong information.

KEY IDEA

@functools.wraps(func) copies the name, docstring and other metadata onto the wrapper. Use it in **every** decorator.

Decorators that take arguments

PYTHON

```python
import functools
def repeat(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def cheer(name):
    print(f"Go {name}!")
cheer("team")
```

1 2 3

OUTPUT

```
Go team!
Go team!
Go team!
```

THREE LAYERS

1. The outer function receives the **settings** and returns a decorator.
2. The decorator receives the **function**.
3. The wrapper receives the **call arguments**.

IN PLAIN ENGLISH

@repeat(times=3) *first calls* **repeat(times=3)**. *Whatever that returns is then used as the decorator.*

TRY IT YOURSELF

Write **@retry(attempts=3)** that calls the function again whenever it raises an exception.

Stacking decorators

PYTHON

```
def bold(func):
    def wrapper(text):
        return f"<b>{func(text)}</b>"
    return wrapper

def italic(func):
    def wrapper(text):
        return f"<i>{func(text)}</i>"
    return wrapper

@bold   # applied last: the outer layer
@italic   # applied first: the inner layer
def title(text):
    return text.title()

print(title("hello world"))
```

OUTPUT

```
<b><i>Hello World</i></b>
```

KEY IDEA

Stacked decorators are applied from the **bottom up**. The code on the left equals **title = bold(italic(title))**.

IN PLAIN ENGLISH

Think of gift wrapping. The decorator nearest the function is the first layer of paper. The top one is the outside layer.

WHERE YOU WILL MEET DECORATORS

- **@property**, **@classmethod**, **@staticmethod**
- **@dataclass**
- **@functools.lru_cache**
- Route and test markers in web and testing frameworks

A class as a decorator

PYTHON

```python
class CountCalls:
    def __init__(self, func):
        self.func = func
        self.calls = 0
    def __call__(self, *args, **kwargs):   # instances are callable
        self.calls += 1
        return self.func(*args, **kwargs)

@CountCalls
def ping():
    return "pong"

ping()
ping()
print(ping(), ping.calls)
print(type(ping).__name__, callable(ping))
```

OUTPUT

```
pong 3
CountCalls True
```

VOCABULARY

Callable: any object that can be called with parentheses. Functions, classes and instances with a **__call__** method all qualify.

KEY IDEA

After decoration, **ping** is an **instance** of **CountCalls**. Calling it runs **__call__**, and the instance can keep state such as a counter.

TIP

Choose a class based decorator when the wrapper needs to remember things between calls.

The functools toolbox

PYTHON

```python
from functools import lru_cache, partial, reduce

@lru_cache(maxsize=None)   # remember earlier results
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(80))
print(fib.cache_info().hits > 0)
square = partial(pow, exp=2)   # freeze some arguments
print(square(9))
from_binary = partial(int, base=2)
print(from_binary("1010"))

print(reduce(lambda acc, n: acc * n, [1, 2, 3, 4, 5]))
```

OUTPUT

```
23416728348467685
True
81
10
120
```

lru_cache

Stores results by argument. Without it **fib(80)** would need longer than a human lifetime. With it the answer is instant.

partial

Builds a new function from an old one with some arguments already filled in.

reduce

Folds a sequence into a single value by applying a two argument function again and again.

Context managers the short way: contextlib

PYTHON

```python
from contextlib import contextmanager, suppress

@contextmanager
def tag(name):
    print(f"<{name}>")   # setup: runs on entry
    try:
        yield name   # the with block runs here
    finally:
        print(f"</{name}>")   # teardown: always runs

with tag("section") as t:
    print("  content inside", t)
with suppress(FileNotFoundError):   # ignore one named error
    open("missing.txt")
print("still running")
```

OUTPUT

```
<section>
  content inside section
</section>
still running
```

KEY IDEA

@contextmanager turns a generator into a context manager. Code before **yield** is the setup. Code after it is the teardown.

TIP

Wrap the **yield** in **try** and **finally** so that the teardown runs even when the block raises.

ALSO IN CONTEXTLIB

suppress ignores chosen exceptions, **redirect_stdout** captures printed text and **ExitStack** manages a varying number of resources.

Quiz: when does a decorator run

PREDICT THE OUTPUT

```
def deco(func):
    print("decorating", func.__name__)
    return func

@deco
def hello():
    print("hello")

print("ready")
hello()
```

ACTUAL OUTPUT

```
decorating hello
ready
hello
```

QUESTION

In which order do the three lines appear?

OPTIONS

A. ready, decorating hello, hello

B. decorating hello, ready, hello

C. ready, hello

D. decorating hello, hello, ready

ANSWER

B. A decorator runs at definition time, as soon as Python reaches the **def**, long before the function is called.

Module 12 recap

1. A decorator takes a function and returns a replacement, usually a wrapper that calls the original.
2. **@name** above a definition is shorthand for **func = name(func)**.
3. A wrapper with ***args** and ****kwargs** fits any function. **functools.wraps** keeps its identity.
4. A decorator with arguments has three layers: settings, function, call.
5. Stacked decorators apply from the bottom up. A class with **__call__** can decorate and keep state.
6. **lru_cache** remembers results, **partial** freezes arguments and **@contextmanager** builds a context manager.

Coming next: Module 13 adds type hints and modern language features.

13

MODULE 13 ADVANCED

Type Hints and Modern Python

Describe the types your code expects, let tools find bugs before the program runs and use the newest syntax.

IN THIS MODULE

1. Annotating functions and variables
2. Unions, optional values and aliases
3. Protocols and generics
4. Typed dictionaries and named tuples
5. Static checking with mypy
6. Advanced pattern matching

Type hints: the basics

PYTHON

```python
def greet(name: str, times: int = 1) -> str:
    return ", ".join([f"Hello {name}"] * times)

age: int = 36   # a variable annotation
price: float = 9.99
names: list[str] = ["Ada", "Alan"]
scores: dict[str, int] = {"Ada": 95}
point: tuple[float, float] = (1.5, 2.0)

print(greet("Ada", 2))
print(greet(42))   # wrong type, yet it runs
print(greet.__annotations__)
```

OUTPUT

```
Hello Ada, Hello Ada
Hello 42
{'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
```

HOW TO READ THEM

- **name: str** says the parameter should be a string.
- The arrow before the colon states the return type.
- **list[str]** is a list whose items are strings.
- **dict[str, int]** maps strings to integers.

KEY IDEA

Hints are documentation that machines can check. The interpreter stores them and otherwise **ignores** them, as line 11 proves.

TIP

Annotate function signatures first. That is where hints give the most value for the least effort.

Unions, optional values and aliases

PYTHON

```python
from typing import Callable, Any

def find(items: list[str], target: str) -> int | None:
    return items.index(target) if target in items else None

def describe(value: int | str | None = None) -> str:
    if value is None:   # the checker narrows the type
        return "nothing"
    return f"{type(value).__name__}: {value}"

type Handler = Callable[[str], None]   # a named alias
def run(callback: Handler, payload: Any) -> None:
    callback(str(payload))

print(find(["a", "b"], "b"), find(["a"], "z"))
print(describe(7), "|", describe("x"), "|", describe())
run(print, 3.5)
```

OUTPUT

```
1 None
int: 7 | str: x | nothing
3.5
```

VOCABULARY OF TYPES

- **A | B** accepts either type.
- **T | None** marks a value that may be missing.
- **Callable[[str], None]** is a function that takes a string and returns nothing.
- **Any** switches checking off for one value.

VOCABULARY

Narrowing: after a test such as **if value is None**, a type checker knows the more precise type inside each branch.

Protocols: duck typing with a safety net

PYTHON

```
from typing import Protocol

class Speaker(Protocol):   # describes a capability
    def speak(self) -> str: ...

class Duck:
    def speak(self) -> str:
        return "Quack"

class Robot:
    def speak(self) -> str:
        return "Beep"

def announce(thing: Speaker) -> None:   # anything with speak()
    print(thing.speak())
announce(Duck())
announce(Robot())
```

OUTPUT

```
Quack
Beep
```

VOCABULARY

Protocol: a class that lists the methods an object must offer. Any class with matching methods satisfies it, without inheriting from it.

KEY IDEA

Duck and **Robot** never mention **Speaker**, yet a type checker accepts both, because their **structure** fits.

NOMINAL VERSUS STRUCTURAL

An abstract base class asks what an object **is**. A protocol asks what it can **do**.

Generics: one definition, many types

PYTHON

```python
def first[T](items: list[T]) -> T:   # T is a type parameter
    return items[0]

class Stack[T]:   # a generic class
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()

names = Stack[str]()
names.push("Ada")
names.push("Alan")
print(first([1.5, 2.5]), names.pop())
# before Python 3.12:  T = TypeVar("T")  from the typing module
```

OUTPUT

```
1.5 Alan
```

VOCABULARY

Generic: code written with a placeholder type that is filled in at each use.

KEY IDEA

first returns whatever type the list holds. Give it a list of floats and the checker knows the result is a float.

IN PLAIN ENGLISH

A stack of T: whatever goes in as T comes out as T.

TIP

Square brackets after a function or class name declare type parameters in Python 3.12 and later.

Typed dictionaries, named tuples and literals

PYTHON

```python
from typing import TypedDict, NamedTuple, Literal, Final

class Movie(TypedDict):   # a dict with known keys
    title: str
    year: int

class Point(NamedTuple):   # a tuple with field names
    x: float
    y: float

Mode = Literal["r", "w", "a"]   # only these exact values
MAX_USERS: Final = 100   # must not be reassigned

film: Movie = {"title": "Alien", "year": 1979}
p = Point(1.5, 2.0)
print(film["title"], p.x, p[1], p._replace(x=9))
print(isinstance(film, dict), isinstance(p, tuple))
```

OUTPUT

```
Alien 1.5 2.0 Point(x=9, y=2.0)
True True
```

FOUR PRECISE TOOLS

- **TypedDict** describes the keys of a dictionary, such as a JSON record.
- **NamedTuple** is an immutable tuple with readable field names.
- **Literal** limits a value to a fixed list.
- **Final** marks a constant.

KEY IDEA

At run time a **Movie** is an ordinary **dict** and a **Point** is an ordinary **tuple**. The extra knowledge exists for the checker.

Static checking with mypy

PYTHON

```python
# file: shop.py
def total(prices: list[float]) -> float:
    return sum(prices)

def label(name: str, qty: int) -> str:
    return name + qty   # bug: str plus int          1

print(total(["4.5", 3]))   # bug: a str in the list   2
```

TERMINAL: mypy shop.py

```
shop.py:6: error: Unsupported operand types for + ("str" and "int")
shop.py:8: error: List item 0 has incompatible type "str"; expected "float"
Found 2 errors in 1 file (checked 1 source file)
```

VOCABULARY

Static type checker: a tool that reads your code **without running it** and reports every place where the types do not fit.

WORKFLOW

1. Install it once with **pip install mypy**.
2. Run it on a file or a whole folder.
3. Fix what it reports, then run the program.

KEY IDEA

Both bugs are found in a second, before a single line executes. Pyright is a popular alternative that also powers editor hints.

Advanced pattern matching

PYTHON

```python
from dataclasses import dataclass

@dataclass
class Circle:
    radius: float
@dataclass
class Rect:
    width: float
    height: float

def area(shape):
    match shape:
        case Circle(radius=r):   # class pattern
            return round(3.14159 * r * r, 2)
        case Rect(w, h) if w == h:   # pattern with a guard
            return f"square {w * h}"
        case Rect(w, h):
            return w * h
        case [x, y, *_]:   # sequence pattern
            return x * y
print(area(Circle(1)), area(Rect(2, 2)), area(Rect(2, 3)), area([5, 6, 7]))
```

OUTPUT

```
3.14 square 4 6 30
```

KINDS OF PATTERN

- A **class pattern** checks the type and pulls out attributes.
- A **guard** adds an **if** condition.
- A **sequence pattern** matches by length and position.
- A **mapping pattern** matches chosen dictionary keys.

KEY IDEA

match tests and unpacks in one step.

What recent Python versions added

Version	Highlights	Example
3.8	Walrus operator, positional only parameters, the equals sign in f strings	**f'{total=}'**
3.9	Dictionary merge operator, built in generic types, **removeprefix**	**a | b**, **list[int]**
3.10	Structural pattern matching, union types with a bar, clearer error messages	**int | None**
3.11	Much faster interpreter, exception groups, **tomllib**, precise error locations	**except* ValueError**
3.12	Type parameter syntax, the **type** statement, more flexible f strings	**def first[T](...)**
3.13	A friendlier interactive shell and an experimental build without the GIL	Coloured tracebacks

TIP

Ask the terminal for your Python version to see what you have. Every example in this tutorial was executed on Python 3.12.

KEY IDEA

Python releases one new version each year, and each version is supported for five years.

Module 13 recap

1. Type hints state the expected types of parameters, return values and variables.
2. The interpreter does not enforce hints. A static checker such as mypy does, before the code runs.
3. **A | B** is a union and **T | None** an optional value. Tests such as **is None** narrow the type.
4. A **Protocol** describes required methods. Generics keep a type consistent through a function or class.
5. **TypedDict**, **NamedTuple**, **Literal** and **Final** describe data precisely.
6. **match** supports class, sequence and mapping patterns as well as guards.

Coming next: Module 14 opens the bonnet: how the interpreter itself works.

14

MODULE 14 ADVANCED

Python Internals

Understand the machinery: objects, attribute lookup, metaclasses, memory, bytecode and the global interpreter lock.

IN THIS MODULE

1. Everything is an object
2. The data model
3. Attribute lookup and descriptors
4. Classes as objects and metaclasses
5. Memory management
6. Bytecode and the GIL

Everything is an object

PYTHON

```
def f():
    pass

for thing in [42, "hi", [1, 2], f, int, None]:
    print(type(thing).__name__, end=" | ")
print()

x = 42
print(id(x) == id(42), type(x), x.bit_length())
print((5).__add__(3), "abc".__len__())   # operators are methods
f.note = "functions accept attributes"
print(f.note, isinstance(int, object), isinstance(f, object))
print(type(int), type(type))
```

OUTPUT

```
int | str | list | function | type | NoneType |
True <class 'int'> 6
8 3
functions accept attributes True True
<class 'type'> <class 'type'>
```

KEY IDEA

Numbers, strings, functions, classes, modules and even **None** are all objects. Each has an **identity**, a **type** and a **value**.

THREE QUESTIONS FOR ANY OBJECT

- **id(obj)**: which object is it?
- **type(obj)**: what kind of object is it?
- **dir(obj)**: what can it do?

UNDER THE HOOD

Classes are objects as well. Their type is **type**, and the type of **type** is **type** itself.

The data model: syntax becomes method calls

You write	Python calls	Purpose
len(x)	**x.__len__()**	Size of a container
x + y	**x.__add__(y)**	Arithmetic and other operators
x[i]	**x.__getitem__(i)**	Indexing and slicing
x.name	**x.__getattribute__('name')**	Attribute access
x(...)	**x.__call__(...)**	Calling an object
for item in x	**x.__iter__()**	Iteration
with x:	**x.__enter__()** and **x.__exit__(...)**	Setup and teardown
print(x)	**x.__str__()**	Text for humans

KEY IDEA

Python syntax is a thin layer over special methods. Implement the method and your object works with the syntax.

VOCABULARY

Data model: the official list of these hooks. Because of their double underscores they are often called dunder methods.

How Python finds an attribute

Writing **obj.name** starts a fixed search. Knowing the order explains properties, methods and many surprises.

1 Data descriptor

A property or similar object on the class wins first.

2 Instance

Next comes the dictionary of the object itself, **obj.__dict__**.

3 Class and parents

Then every class in the MRO is searched for methods and class attributes.

4 Fallback

If **__getattr__** exists it is called. Otherwise **AttributeError** is raised.

UNDER THE HOOD

Functions stored on a class are descriptors too. When found through an instance they bind themselves to it, and that is how **self** gets filled in.

TIP

vars(obj) shows the instance dictionary. **type(obj).__mro__** shows the classes that will be searched.

Descriptors: reusable attribute logic

PYTHON

```python
class Positive:
    def __set_name__(self, owner, name):
        self.name = "_" + name
    def __get__(self, obj, objtype=None):
        return getattr(obj, self.name)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self.name[1:]} must be positive")
        setattr(obj, self.name, value)

class Order:
    quantity = Positive()   # one descriptor per attribute
    price = Positive()
    def __init__(self, quantity, price):
        self.quantity = quantity
        self.price = price

o = Order(3, 9.5)
print(o.quantity * o.price)
o.price = 0
```

OUTPUT

```
28.5
ValueError: price must be positive
```

VOCABULARY

Descriptor: an object stored on a class that defines **__get__**, **__set__** or **__delete__** and so takes control of one attribute.

KEY IDEA

A property validates one attribute of one class. A descriptor packages the same rule so that many attributes and classes can share it.

Dynamic attributes: __getattr__ and __setattr__

PYTHON

```
class Config:
    def __init__(self, **values):
        self._values = values
    def __getattr__(self, name):   # only after lookup fails
        if name in self._values:
            return self._values[name]
        raise AttributeError(f"no setting named {name!r}")

cfg = Config(host="localhost", port=8080)
print(cfg.host, cfg.port)

class Tracer:
    def __setattr__(self, name, value):   # every assignment
        print(f"  setting {name} = {value}")
        super().__setattr__(name, value)

t = Tracer()
t.x = 1
print(getattr(t, "x"), getattr(t, "y", "default"), hasattr(t, "y"))
```

OUTPUT

```
localhost 8080
  setting x = 1
1 default False
```

TWO HOOKS

- **__getattr__** runs only when the normal search finds nothing.
- **__setattr__** runs on **every** assignment to an attribute.

TIP

getattr, **setattr** and **hasattr** work with attribute names held in strings.

Classes are objects, built by type

PYTHON

```
class Dog:
    sound = "Woof"

print(type(Dog), type(Dog()))   # the class of a class is type

def speak(self):
    return self.sound * 2

Cat = type("Cat", (object,), {"sound": "Meow", "speak": speak})
print(Cat().speak(), Cat.__name__, Cat.__bases__)

Dog.speak = speak   # add a method afterwards
print(Dog().speak())
print(Dog.__dict__["sound"], Dog.__mro__)
```

OUTPUT

```
<class 'type'> <class '__main__.Dog'>
MeowMeow Cat (<class 'object'>,)
WoofWoof
Woof (<class '__main__.Dog'>, <class 'object'>)
```

KEY IDEA

A **class** statement is a convenient way of calling **type(name, bases, namespace)**. Line 9 builds a class with no **class** keyword at all.

UNDER THE HOOD

Python runs the class body as ordinary code, collects the names it defines in a dictionary and hands that dictionary to **type**.

WATCH OUT

Adding methods to a class from outside is called monkey patching. It is useful in tests and confusing almost everywhere else.

Metaclasses: classes that build classes

PYTHON

```
class Registry(type):   # inherits from type
    plugins = {}
    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if bases:   # skip the base class itself
            mcls.plugins[name.lower()] = cls
        return cls

class Plugin(metaclass=Registry):
    pass
class Csv(Plugin):
    pass
class Json(Plugin):
    pass

print(sorted(Registry.plugins))
print(type(Csv).__name__, isinstance(Csv, Registry))
```

OUTPUT

```
['csv', 'json']
Registry True
```

VOCABULARY

Metaclass: the class of a class. It controls how classes themselves are created.

IN PLAIN ENGLISH

Each time a subclass of **Plugin** *is defined,* **Registry.__new__** *runs and files the new class in a dictionary.*

WATCH OUT

Metaclasses are powerful and rarely needed. Reach for a decorator or **__init_subclass__** first.

Lighter tools: __init_subclass__ and class decorators

PYTHON

```python
class Plugin:
    registry = {}
    def __init_subclass__(cls, /, kind, **kwargs):   # per subclass
        super().__init_subclass__(**kwargs)
        cls.kind = kind
        Plugin.registry[kind] = cls

class CsvPlugin(Plugin, kind="csv"):
    pass
class JsonPlugin(Plugin, kind="json"):
    pass
print(sorted(Plugin.registry), JsonPlugin.kind)

def add_repr(cls):   # a class decorator
    cls.__repr__ = lambda self: f"<{cls.__name__}>"
    return cls

@add_repr
class Token:
    pass
print(Token())
```

OUTPUT

```
['csv', 'json'] json
<Token>
```

KEY IDEA

__init_subclass__ runs in the **parent** whenever a subclass is created. It covers most uses of metaclasses with far less machinery.

CLASS DECORATORS

A class decorator receives the finished class and may modify or replace it. **@dataclass** works this way.

Memory: reference counts and the cycle collector

PYTHON

```
import sys, gc, weakref

data = [1, 2, 3]
alias = data
print(sys.getrefcount(data) - 1)   # two names share the list
del alias
print(sys.getrefcount(data) - 1)

class Node:
    def __init__(self):
        self.partner = None
a, b = Node(), Node()
a.partner, b.partner = b, a   # a reference cycle
watcher = weakref.ref(a)   # observe without owning
del a, b
print(watcher() is not None)   # the cycle keeps both alive
gc.collect()   # the cycle detector runs
print(watcher() is None)
```

OUTPUT

```
2
1
True
True
```

VOCABULARY

Reference count: the number of references that point at an object. When it reaches zero, CPython frees the object at once.

KEY IDEA

Two objects that refer to each other never reach zero. A separate garbage collector finds and frees such cycles.

VOCABULARY

Weak reference: a reference that does not keep its target alive.

Identity surprises: caching and interning

PYTHON

```
a = 256
b = 256
print(a is b)   # small integers are cached
x = int("1000")
y = int("1000")
print(x == y, x is y)   # equal, yet two objects
s1 = "hello"
s2 = "hello"
print(s1 is s2)   # identical literals are shared
t1 = "".join(["hel", "lo"])
print(t1 == s1, t1 is s1)   # built at run time: a new object
import sys
print(sys.intern(t1) is s1)   # interning on request
```

OUTPUT

```
True
True False
True
True False
True
```

UNDER THE HOOD

CPython keeps one shared object for each small integer and for many string literals. This saves memory and is safe because both types are immutable.

WATCH OUT

These are implementation details and may differ between versions. Never write code that depends on them.

KEY IDEA

The lesson stays the same as in Module 3: compare values with ==. Keep **is** for **None**.

A look at bytecode with dis

PYTHON

```
import dis

def add_tax(price):
    return price * 1.2

dis.dis(add_tax)
print(add_tax.__code__.co_varnames, add_tax.__code__.co_consts)
```

BYTECODE

```
  3           0 RESUME                   0

  4           2 LOAD_FAST                0 (price)
              4 LOAD_CONST               1 (1.2)
              6 BINARY_OP                5 (*)
             10 RETURN_VALUE
('price',) (None, 1.2)
```

VOCABULARY

Bytecode: the compact instructions that the Python virtual machine executes. One line of source usually becomes several instructions.

READING THE LISTING

- **LOAD_FAST** pushes a local variable onto a stack.
- **LOAD_CONST** pushes a constant.
- **BINARY_OP** pops two values and pushes the result.
- **RETURN_VALUE** hands the top of the stack to the caller.

TIP

You rarely need bytecode in daily work. It is, however, the best way to settle arguments about what Python really does.

The global interpreter lock

TWO THREADS, ONE PROCESS

Thread A	**runs**	waits	**runs**	waits	**runs**	waits
Thread B	waits	**runs**	waits	**runs**	waits	**runs**

One lock is shared, so the threads take turns.

TWO PROCESSES

Process 1	**runs**	**runs**	**runs**	**runs**	**runs**	**runs**
Process 2	**runs**	**runs**	**runs**	**runs**	**runs**	**runs**

Each process has its own interpreter and its own lock.

VOCABULARY

GIL: a lock inside CPython that lets only one thread execute Python bytecode at any moment.

KEY IDEA

Threads still help when a program **waits** for a network or a disk, because a waiting thread releases the lock. For heavy computation use processes.

UNDER THE HOOD

The lock keeps reference counting simple and fast. Python 3.13 offers an experimental build that removes it.

Module 14 recap

1. Every value is an object with an identity, a type and a value. Classes are objects whose type is **type**.
2. Operators and built in functions are translated into special method calls: the data model.
3. Attribute lookup checks data descriptors, the instance, the classes in the MRO and finally **__getattr__**.
4. Descriptors, **__init_subclass__**, class decorators and metaclasses customise classes, from light to heavy.
5. CPython frees objects by reference counting and uses a collector for reference cycles.
6. Source is compiled to bytecode. The GIL lets one thread run bytecode at a time.

Coming next: Module 15 uses this knowledge to run many tasks at once.

15

MODULE 15 ADVANCED

Concurrency

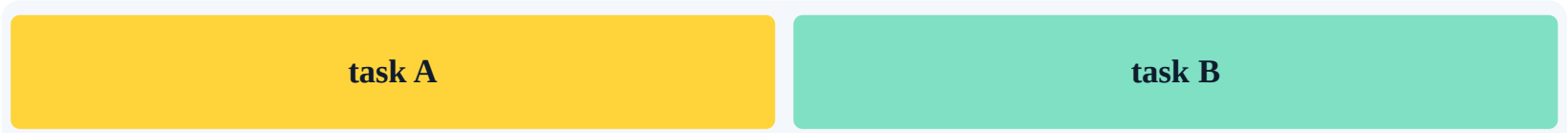
Do several things at once: threads for waiting, processes for computing and asyncio for thousands of connections.

IN THIS MODULE

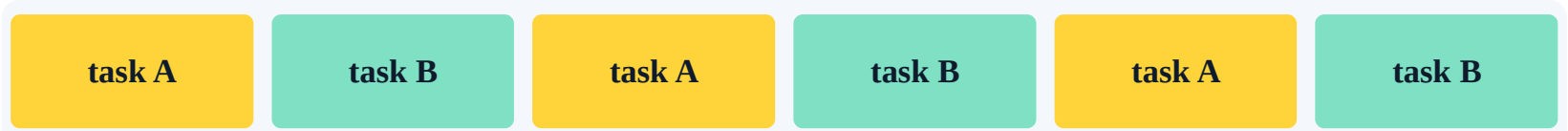
1. Concurrency versus parallelism
2. Threads, races and locks
3. Thread pools and process pools
4. The asyncio event loop
5. async, await, tasks and timeouts
6. Choosing the right model

Concurrency versus parallelism

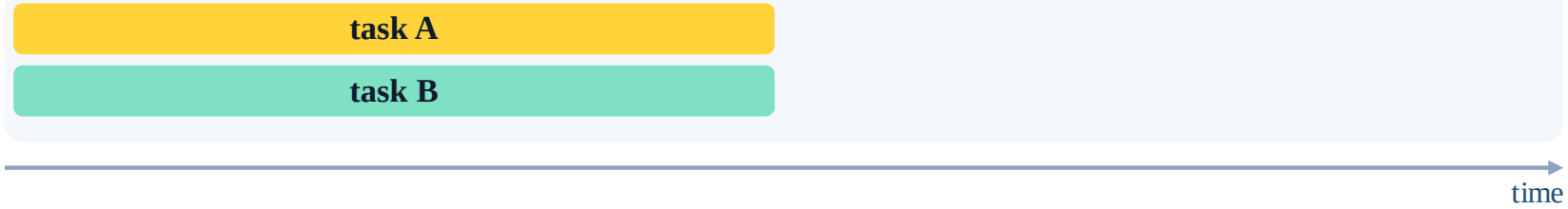
Sequential
One task finishes before the next begins



Concurrent
Tasks take turns on a single core



Parallel
Tasks run at the same instant on two cores



VOCABULARY

Concurrency: making progress on several tasks during the same period by switching between them.

VOCABULARY

Parallelism: executing several tasks at literally the same moment on separate processor cores.

KEY IDEA

I/O bound work mostly waits for a disk or a network and gains from concurrency. **CPU bound** work mostly computes and needs parallelism.

Threads: waiting in parallel

PYTHON

```python
import threading, time

def download(name, seconds):
    print(f"start {name}")
    time.sleep(seconds)   # stands in for network waiting
    print(f"done  {name}")

begin = time.perf_counter()
jobs = [("A", 0.3), ("B", 0.1), ("C", 0.2)]
threads = [threading.Thread(target=download, args=job) for job in jobs]
for t in threads:
    t.start()   # begin running
for t in threads:
    t.join()   # wait until it has finished
elapsed = time.perf_counter() - begin
print(f"total under half a second: {elapsed < 0.5}")
```

OUTPUT

```
start A
start B
start C
done  B
done  C
done  A
total under half a second: True
```

VOCABULARY

Thread: a separate line of execution inside one program. All threads share the same memory.

KEY IDEA

The three waits overlap. The total time is that of the slowest job, about 0.3 seconds, not the sum of 0.6.

Race conditions: when threads collide

PYTHON

```python
import threading, time

balance = 100

def withdraw(amount):
    global balance
    current = balance    # 1. read
    time.sleep(0.01)    # 2. another thread sneaks in
    balance = current - amount    # 3. write a stale result

threads = [threading.Thread(target=withdraw, args=(30,))
           for _ in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()
print("balance:", balance, "| expected: 10")
```

1 2 3

OUTPUT

```
balance: 70 | expected: 10
```

VOCABULARY

Race condition: a bug whose outcome depends on the unlucky timing of threads that touch the same data.

IN PLAIN ENGLISH

All three threads read 100 before any of them writes. Each then stores 70, and two withdrawals vanish.

WATCH OUT

Races are the hardest bugs to find. They may appear once in a million runs and never while you are watching.

Locks: one thread at a time

PYTHON

```python
import threading, time

balance = 100
lock = threading.Lock()
def withdraw(amount):
    global balance
    with lock:   # only one thread may enter
        current = balance
        time.sleep(0.01)
        balance = current - amount
threads = [threading.Thread(target=withdraw, args=(30,))
           for _ in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()
print("balance:", balance)
```

1

OUTPUT

```
balance: 10
```

VOCABULARY

Lock: an object that only one thread can hold at a time. Every other thread waits at the **with** line until it is released.

KEY IDEA

Protect every read and modify and write sequence on shared data with the **same** lock.

TIP

Better still, share nothing. Hand work to threads through a **queue.Queue**, which does its own locking.

Thread pools

PYTHON

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def fetch(page):
    time.sleep(0.1 * page)   # pretend to download
    return f"page {page}: {page * 100} bytes"
with ThreadPoolExecutor(max_workers=4) as pool:
    for line in pool.map(fetch, [3, 1, 2]):   # input order
        print(line)
    futures = [pool.submit(fetch, p) for p in (3, 1, 2)]
    for future in as_completed(futures):   # finishing order
        print("finished:", future.result())
```

OUTPUT

```
page 3: 300 bytes
page 1: 100 bytes
page 2: 200 bytes
finished: page 1: 100 bytes
finished: page 2: 200 bytes
finished: page 3: 300 bytes
```

VOCABULARY

Pool: a fixed team of worker threads that take jobs from a queue. You submit work and collect results.

VOCABULARY

Future: a placeholder for a result that is not ready yet. **result()** waits for it and raises any exception from the worker.

KEY IDEA

pool.map returns results in input order. **as_completed** yields each future as soon as it is done.

Process pools: true parallelism

PYTHON

```python
from concurrent.futures import ProcessPoolExecutor

def count_primes(limit):
    return sum(all(n % d for d in range(2, int(n ** 0.5) + 1))
               for n in range(2, limit))

if __name__ == "__main__":  # required with processes
    limits = [20_000, 30_000, 40_000]
    with ProcessPoolExecutor() as pool:
        results = pool.map(count_primes, limits)
        for limit, total in zip(limits, results):
            print(f"primes below {limit}: {total}")
```

OUTPUT

```
primes below 20000: 2262
primes below 30000: 3245
primes below 40000: 4203
```

KEY IDEA

Each worker is a separate Python process with its own interpreter and its own GIL, so CPU bound work really runs on several cores.

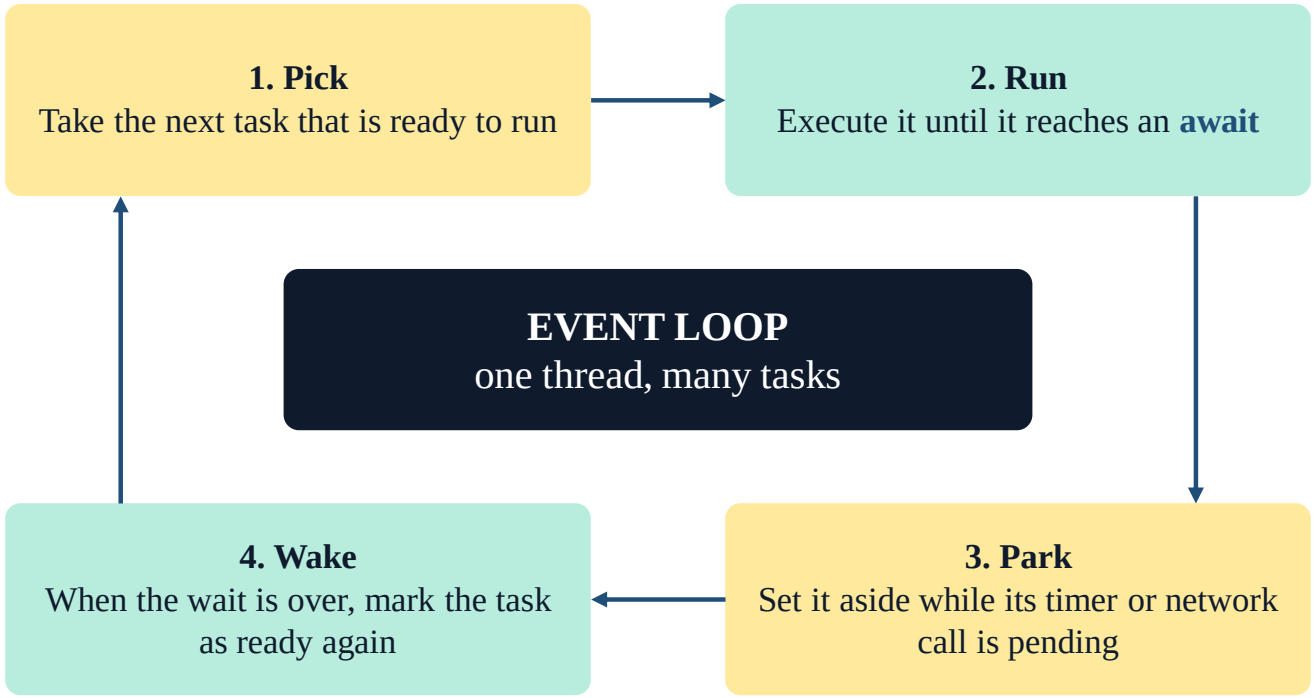
THE PRICE

- Starting a process is slow compared with a thread.
- Arguments and results are pickled and copied between processes.
- Workers do not share variables.

TIP

The interface matches **ThreadPoolExecutor**. Switching between threads and processes is a one word change.

asyncio: the event loop



WATCH OUT

A blocking call such as **time.sleep()** freezes the whole loop. Always use the awaitable version, **await asyncio.sleep()**.

VOCABULARY

Coroutine: a function defined with **async def**. It can pause at **await** and continue later from the same spot.

VOCABULARY

Event loop: the scheduler that runs coroutines one at a time and switches between them whenever one waits.

KEY IDEA

await is a polite pause. It tells the loop: I am waiting, let someone else run. Switching happens **only** at these points.

UNDER THE HOOD

No threads are involved, so there are no race conditions between two lines of ordinary code.

async and await

PYTHON

```python
import asyncio

async def brew(drink, seconds):
    print(f"start {drink}")
    await asyncio.sleep(seconds)   # hand control to the loop
    print(f"ready {drink}")
    return drink

async def main():
    results = await asyncio.gather(brew("tea", 0.2), brew("coffee", 0.1))
    print(results)

asyncio.run(main())   # start the event loop
```

1 2 3 4

OUTPUT

```
start tea
start coffee
ready coffee
ready tea
['tea', 'coffee']
```

LINE BY LINE

1. **async def** defines a coroutine function.
2. **await** pauses this coroutine until the awaited thing is finished.
3. **gather** runs several coroutines concurrently and returns their results in order.
4. **asyncio.run** creates the loop, runs **main** and closes the loop.

WATCH OUT

Calling **brew('tea', 1)** without **await** runs nothing. It only creates a coroutine object, and Python warns that it was never awaited.

Tasks: sequential versus concurrent awaiting

PYTHON

```python
import asyncio, time

async def fetch(name, delay):
    await asyncio.sleep(delay)
    return name

async def main():
    t0 = time.perf_counter()
    one = await fetch("first", 0.2)   # one after the other
    two = await fetch("second", 0.2)
    print(one, two, round(time.perf_counter() - t0, 1), "seconds")

    t0 = time.perf_counter()
    a = asyncio.create_task(fetch("third", 0.2))   # both start now
    b = asyncio.create_task(fetch("fourth", 0.2))
    print(await a, await b, round(time.perf_counter() - t0, 1), "seconds")

asyncio.run(main())
```

OUTPUT

```
first second 0.4 seconds
third fourth 0.2 seconds
```

KEY IDEA

Awaiting coroutines one by one is **sequential**. Wrapping them in tasks schedules them at once, so their waits overlap.

VOCABULARY

Task: a coroutine that has been handed to the event loop and is already running in the background.

Task groups, semaphores and timeouts

PYTHON

```python
import asyncio

async def job(n, limit):
    async with limit:   # at most two at a time
        await asyncio.sleep(0.1)
        return n * n

async def main():
    limit = asyncio.Semaphore(2)
    async with asyncio.TaskGroup() as group:
        tasks = [group.create_task(job(n, limit)) for n in range(4)]
    print([t.result() for t in tasks])   # all finished here

    try:
        async with asyncio.timeout(0.05): # give up after 50 ms
            await asyncio.sleep(1)
    except TimeoutError:
        print("too slow, cancelled")

asyncio.run(main())
```

OUTPUT

```
[0, 1, 4, 9]
too slow, cancelled
```

THREE SAFETY TOOLS

- A **TaskGroup** waits for all of its tasks and cancels the rest if one fails.
- A **Semaphore** limits how many tasks run a section at once.
- A **timeout** cancels work that takes too long.

TIP

Use a semaphore to stay polite to a server: never more than a handful of requests at a time.

Choosing a concurrency model

Question	threading	multiprocessing	asyncio
Best for	I/O bound work with ordinary blocking libraries	CPU bound computation	Very many I/O bound tasks
Uses	One core at a time	Several cores	One core and one thread
Switching happens	At any moment, decided by the system	Processes are independent	Only at an **await**
Cost per task	Moderate	High	Tiny
Main risk	Race conditions	Copying data between processes	One blocking call stalls everything
Start with	**ThreadPoolExecutor**	**ProcessPoolExecutor**	**asyncio.run()**

KEY IDEA

Ask what your program spends its time on. **Waiting**: threads or asyncio. **Computing**: processes.

TIP

Measure first. Many programs are fast enough with no concurrency at all, and simple code is easier to trust.

Quiz: the order of coroutines

PREDICT THE OUTPUT

```
import asyncio

async def step(name):
    print(name, "start")
    await asyncio.sleep(0)
    print(name, "end")

async def main():
    await asyncio.gather(step("A"), step("B"))

asyncio.run(main())
```

ACTUAL OUTPUT

```
A start
B start
A end
B end
```

QUESTION

In which order do the four lines appear?

OPTIONS

A. A start, A end, B start, B end

B. A start, B start, A end, B end

C. The order changes from run to run

D. B start, A start, B end, A end

ANSWER

B. Each coroutine runs until its **await**, which hands control to the loop. The loop then starts the other one.

Module 15 recap

1. Concurrency is taking turns. Parallelism is running at the same instant on several cores.
2. Threads share memory and suit I/O bound work. Guard shared data with a lock to prevent races.
3. **ThreadPoolExecutor** and **ProcessPoolExecutor** offer the same simple interface of **map** and **submit**.
4. Processes bypass the GIL and suit CPU bound work, at the price of start time and data copying.
5. **async def** defines a coroutine. **await** pauses it and lets the event loop run another task.
6. **gather**, tasks and task groups run coroutines concurrently. Never block the event loop.

Coming next: Module 16 keeps code correct and fast: testing, debugging and performance.

16

MODULE 16 ADVANCED

Testing, Debugging and Performance

Prove that code works, find out why it does not and make it fast where it matters.

IN THIS MODULE

1. Unit tests with pytest
2. Fixtures, parameters and mocks
3. The debugger and logging
4. Measuring speed
5. Performance idioms
6. Collections, regex and dates
7. Style and tooling

Unit tests with pytest

FILE: pricing.py

```
def apply_discount(price, percent):
    if not 0 <= percent <= 100:
        raise ValueError("percent must be 0 to 100")
    return round(price * (1 - percent / 100), 2)
```

VOCABULARY

Unit test: a small function that calls your code with known inputs and asserts the expected result.

KEY IDEA

pytest finds files named **test_*.py**, runs every function whose name starts with **test_** and explains each failed **assert** in detail.

TIP

Each dot in the report is one passing test. Run the tests after **every** change.

FILE: test_pricing.py

```
import pytest
from pricing import apply_discount

def test_ten_percent_off():
    assert apply_discount(200, 10) == 180

def test_zero_keeps_price():
    assert apply_discount(59.99, 0) == 59.99

def test_rejects_bad_percent():
    with pytest.raises(ValueError):
        apply_discount(100, 150)
```

TERMINAL: pytest

```
collected 3 items
test_pricing.py ...
3 passed in 0.01s
```

Parametrised tests and fixtures

PYTHON

```python
import pytest

@pytest.mark.parametrize("text, expected", [
    ("level", True),
    ("Python", False),
    ("", True),
])
def test_palindrome(text, expected):   # runs once per row
    assert (text == text[::-1]) is expected

@pytest.fixture
def cart():   # fresh data per test
    return {"apple": 2, "pear": 1}

def test_cart_total(cart):   # requested by name
    assert sum(cart.values()) == 3

def test_cart_is_fresh(cart, tmp_path):   # tmp_path is built in
    cart.clear()
    assert tmp_path.exists()
```

TERMINAL: pytest

```
5 passed in 0.02s
```

PARAMETRIZE

One test function, many input rows. Each row is reported as its own test.

VOCABULARY

Fixture: a function that prepares what a test needs. A test receives it simply by naming it as a parameter.

TIP

tmp_path is a ready made fixture that gives every test its own empty folder.

Replacing dependencies with mocks

PYTHON

```python
from unittest.mock import Mock, patch
import random

def roll_twice():
    return random.randint(1, 6) + random.randint(1, 6)

with patch("random.randint", return_value=4) as fake:   # swap it out
    print(roll_twice(), fake.call_count)

mailer = Mock()   # a stand in object
mailer.send.return_value = "queued"
def notify(service, user):
    return service.send(to=user, subject="Welcome")

print(notify(mailer, "ada@example.com"))
mailer.send.assert_called_once_with(to="ada@example.com", subject="Welcome")
print(mailer.send.call_args.kwargs["to"])
```

OUTPUT

```
8 2
queued
ada@example.com
```

VOCABULARY

Mock: a fake object that accepts any call, records it and returns what you tell it to.

KEY IDEA

Mocks keep tests fast and repeatable by replacing randomness, clocks, networks and databases.

PATCH

patch swaps a real object for a mock inside a **with** block and restores the original afterwards.

The debugger: breakpoint and pdb

PYTHON

```python
def average(values):
    total = sum(values)
    breakpoint()   # execution pauses here
    return total / len(values)

average([3, 4, 8])
```

1

DEBUGGER SESSION

```
> demo.py(4)average()
(Pdb) p total
15
(Pdb) p len(values)
3
(Pdb) n
(Pdb) c
```

ESSENTIAL COMMANDS

- **p expr** prints the value of an expression.
- **n** runs the next line.
- **s** steps into a function call.
- **c** continues to the next breakpoint.
- **l** lists the source around the current line.
- **w** shows the call stack.
- **q** quits the debugger.

KEY IDEA

A debugger lets you stop time, look at every variable and move forward one line at a time. It beats a hundred **print** calls.

Logging instead of print

PYTHON

```
import logging, sys

logging.basicConfig(level=logging.INFO, stream=sys.stdout,
                    format="%(levelname)s %(name)s: %(message)s")
log = logging.getLogger("shop")

def checkout(cart):
    log.debug("cart content: %s", cart)   # hidden at INFO level
    if not cart:
        log.warning("checkout with an empty cart")
        return 0
    total = sum(cart.values())
    log.info("order placed, total %.2f", total)
    return total

checkout({"tea": 2.5, "cake": 4.0})
checkout({})
log.error("payment failed for order %s", 42)
```

OUTPUT

```
INFO shop: order placed, total 6.50
WARNING shop: checkout with an empty cart
ERROR shop: payment failed for order 42
```

FIVE LEVELS

DEBUG, **INFO**, **WARNING**, **ERROR**, **CRITICAL**. The configured level decides which messages are shown.

KEY IDEA

Logging can be switched on, off or redirected to a file without touching the code that produces the messages.

TIP

Inside an **except** block, **log.exception()** records the message together with the full traceback.

Measuring speed: timeit and cProfile

PYTHON

```
import timeit

loop = """
result = []
for n in range(1000):
    result.append(n * 2)
"""
comp = "result = [n * 2 for n in range(1000)]"
t_loop = timeit.timeit(loop, number=2000)
t_comp = timeit.timeit(comp, number=2000)
print("comprehension is faster:", t_comp < t_loop)

import cProfile, pstats, io
def slow():
    return sorted(str(n) for n in range(50_000))
profiler = cProfile.Profile()
profiler.runcall(slow)
stats = pstats.Stats(profiler, stream=io.StringIO())
print("calls recorded:", stats.total_calls > 50_000)
```

OUTPUT

```
comprehension is faster: True
calls recorded: True
```

TWO TOOLS

- **timeit** runs a small snippet many times and reports the total time.
- **cProfile** records every function call in a real program and shows where the time goes.

KEY IDEA

Never guess where a program is slow. Measure, fix the biggest item and measure again.

Big O in practice

PYTHON

```python
import timeit

big_list = list(range(100_000))
big_set = set(big_list)
t_list = timeit.timeit(lambda: 99_999 in big_list, number=200)
t_set = timeit.timeit(lambda: 99_999 in big_set, number=200)
print(f"set at least 100 times faster: {t_list > 100 * t_set}")

def has_duplicates_slow(items):   # O(n squared)
    return any(items[i] in items[i + 1:] for i in range(len(items)))

def has_duplicates_fast(items):   # O(n)
    return len(set(items)) != len(items)

data = list(range(2_000))
print(has_duplicates_slow(data), has_duplicates_fast(data))
```

OUTPUT

```
set at least 100 times faster: True
False False
```

KEY IDEA

Choosing the right data structure or algorithm can make code thousands of times faster. No amount of small tuning competes with that.

IN PLAIN ENGLISH

The slow version compares every item with every later item. The fast version lets a set do the work in a single pass.

TRY IT YOURSELF

Time both duplicate checks with **timeit** for 2 000 and for 4 000 items. How does each one grow?

Performance idioms worth knowing

1 Join strings, do not add them

Adding in a loop copies the text again and again.

```
text = "".join(parts)
```

2 Look things up in sets and dicts

Membership in a list is a slow scan.

```
if user_id in valid_ids:
```

3 Let built in functions loop

They run in C and beat a hand written loop.

```
total = sum(prices)
```

4 Generate, do not store

A generator avoids building a temporary list.

```
sum(n * n for n in numbers)
```

5 Cache repeated work

Remember results of pure, expensive functions.

```
@functools.lru_cache
```

6 Measure before you optimise

Profile first, then fix only the hot spot.

```
python -m cProfile app.py
```

KEY IDEA

Write clear code first. Optimise only when measurements prove that a specific part is too slow.

The collections toolbox

PYTHON

```
from collections import Counter, defaultdict, deque, namedtuple

votes = Counter("abracadabra")
print(votes.most_common(2), votes["z"])
groups = defaultdict(list)   # missing keys start empty
for word in ["apple", "avocado", "banana"]:
    groups[word[0]].append(word)
print(dict(groups))
queue = deque([1, 2, 3], maxlen=3)   # fast at both ends
queue.appendleft(0)
queue.append(9)
print(list(queue), queue[0])
Point = namedtuple("Point", "x y")
print(Point(1, 2), Point(1, 2).x)
```

OUTPUT

```
[('a', 5), ('b', 2)] 0
{'a': ['apple', 'avocado'], 'b': ['banana']}
[1, 2, 9] 1
Point(x=1, y=2) 1
```

FOUR SPECIALISTS

- **Counter** counts hashable items.
- **defaultdict** creates missing values on first access.
- **deque** adds and removes at both ends in constant time.
- **namedtuple** is a light tuple with named fields.

TIP

A **deque** with **maxlen** is a ready made sliding window: old items fall off the far end.

Regular expressions

PYTHON

```
import re

text = "Order 66 shipped on 2026/10/06 to ada@example.com"
print(re.findall(r"\d+", text))   # every run of digits
print(re.search(r"(\d{4})/(\d{2})/(\d{2})", text).groups())
match = re.search(r"(?P<user>\w+)@(?P<host>[\w.]+)", text)
print(match["user"], match["host"], match.span())
print(re.sub(r"\d", "#", "PIN 4821"))   # replace
print(re.split(r"[,;]\s*", "a, b;c,   d"))   # split on a pattern
pattern = re.compile(r"^\w+ \d+$")   # compile once
print(bool(pattern.match("Order 66")), bool(pattern.match("Order x")))
```

OUTPUT

```
['66', '2026', '10', '06']
('2026', '10', '06')
ada example.com (34, 49)
PIN ####
['a', 'b', 'c', 'd']
True False
```

THE CORE SYMBOLS

- **\d** digit, **\w** word character, **\s** whitespace
- **+** one or more, ***** zero or more, **?** optional
- **{4}** exactly four times
- **( )** capture a group, **[ ]** any one of a set
- **^** start and **$** end of the text

TIP

Always write patterns as raw strings with an **r** prefix, so that backslashes reach the regex engine untouched.

Dates and times

PYTHON

```
from datetime import datetime, date, timedelta

launch = datetime(2026, 10, 6, 14, 30)
print(launch.strftime("%A %d %B %Y, %H:%M"))
print(launch.year, launch.month, launch.weekday())

deadline = launch + timedelta(days=45, hours=3)   # date arithmetic
print(deadline.strftime("%d %b %Y %H:%M"))
print((date(2027, 1, 1) - launch.date()).days, "days to the new year")
parsed = datetime.strptime("25/12/2026 08:00", "%d/%m/%Y %H:%M")
print(parsed > launch, parsed.strftime("day %j of the year"))
```

OUTPUT

```
Tuesday 06 October 2026, 14:30
2026 10 1
20 Nov 2026 17:30
87 days to the new year
True day 359 of the year
```

THE MAIN TYPES

- **date**: year, month, day.
- **datetime**: a date plus a time.
- **timedelta**: a duration. Subtracting two dates gives one.

TIP

Remember **f** for format and **p** for parse: **strftime** turns a date into text and **strptime** turns text into a date.

WATCH OUT

A **datetime** without a time zone is called naive. For real schedules attach a zone from the **zoneinfo** module.

Style and tooling

PEP 8 is the official style guide. The first three cards summarise it and the last three name tools that enforce it for you.

Naming

snake_case for functions and variables, **PascalCase** for classes, **UPPER_CASE** for constants.

Layout

Four spaces per level, lines of at most 79 to 99 characters and two blank lines between top level definitions.

Imports

At the top of the file, grouped: standard library, installed packages, your own modules.

Formatter

Black or Ruff rewrite your code into one consistent style, so nobody argues about spacing.

Linter

Ruff or Flake8 flag unused imports, undefined names and suspicious constructs within milliseconds.

Type checker

mypy or Pyright verify your type hints across the whole project.

KEY IDEA

Code is read far more often than it is written. A consistent style lets readers focus on what the code does.

TIP

Run the formatter, the linter, the type checker and the tests automatically before every commit.

Module 16 recap

1. A test is a function with assertions. pytest discovers and runs every **test_** function.
2. Fixtures prepare data, **parametrize** multiplies cases and mocks replace slow or random dependencies.
3. **breakpoint()** pauses a program so that you can inspect it. **logging** records what happened.
4. **timeit** measures snippets and **cProfile** finds the slow function. Measure before optimising.
5. The biggest wins come from better data structures and algorithms.
6. **collections**, **re** and **datetime** solve everyday problems. Formatters and linters keep code clean.

Coming next: Module 17 combines everything in one complete project.

17

MODULE 17 ADVANCED

Capstone and Next Steps

Combine everything into one complete, tested command line application, then choose where to go next.

IN THIS MODULE

1. The project brief
2. Designing in layers
3. Model, storage and rules
4. The command line interface
5. Tests for the whole project
6. Where to go from here

Capstone: an expense tracker for the terminal

A capstone project proves that the separate topics of this tutorial work together. Type every step yourself and run it before you move on.

1 The goal

A command line tool that records what you spend and reports the totals for each category.

2 The commands

expenses add 4.5 food coffee stores one entry. **expenses summary** prints the totals.

3 The skills

Dataclasses, JSON files, pathlib, argparse, exceptions, comprehensions and pytest.

4 The plan

Four small files of code and one file of tests, built in six steps. Each step runs on its own.

KEY IDEA

Real programs are assembled from small pieces that are easy to test. No function in this project is longer than twelve lines.

TRY IT YOURSELF

Before you turn the page, write down which functions you think the program needs. Compare your list with the design on the next slide.

Designing the project in layers

cli.py: the command line
argparse turns typed words into function calls

service.py: the rules
add an expense, group by category, build the summary

storage.py: persistence
load and save the list of expenses as JSON

expenses.json: the data file
plain text that survives between runs

models.py
the **Expense** dataclass
used by every layer

KEY IDEA

Each layer talks only to the layer directly below it.

TIP

The rules never print and never open files, so they are trivial to test.

VOCABULARY

Separation of concerns: every file has one job and one reason to change.

Step 1: the data model

PYTHON

```python
# models.py
from dataclasses import dataclass, asdict
from datetime import date

@dataclass(frozen=True)
class Expense:
    amount: float
    category: str
    note: str = ""
    day: str = ""

    def __post_init__(self):
        if self.amount <= 0:
            raise ValueError("amount must be positive")
        if not self.day:
            today = date.today().strftime("%Y/%m/%d")
            object.__setattr__(self, "day", today)

e = Expense(4.5, "food", "coffee", "2026/10/06")
print(e)
print(asdict(e)["category"], e.day)
```

OUTPUT

```
Expense(amount=4.5, category='food', note='coffee',
day='2026/10/06')
food 2026/10/06
```

DESIGN CHOICES

- **frozen=True**: an expense never changes after it is recorded.
- **__post_init__** validates the amount, so an invalid expense cannot exist.
- A missing day is filled with the current date.

UNDER THE HOOD

A frozen dataclass blocks normal assignment, so the initialiser sets the day through **object.__setattr__**.

Step 2: storage in a JSON file

PYTHON

```python
# storage.py
import json
from pathlib import Path

def load(path):
    if not path.exists():
        return []   # first run: nothing saved
    with path.open(encoding="utf8") as f:
        return [Expense(**row) for row in json.load(f)]

def save(path, expenses):
    with path.open("w", encoding="utf8") as f:
        json.dump([asdict(e) for e in expenses], f, indent=2)

db = Path("expenses.json")
save(db, [Expense(4.5, "food", "coffee", "2026/10/06")])
print(load(db))
```

OUTPUT

```
[Expense(amount=4.5, category='food', note='coffee', day='2026/10/06')]
```

HOW IT WORKS

- **asdict** turns each expense into a plain dictionary that JSON accepts.
- **Expense(**row)** unpacks a dictionary back into an object and validates it again.

KEY IDEA

Saving and then loading returns equal objects. This round trip is the first thing the tests will check.

TIP

The functions receive the path as a parameter. Tests can therefore point them at a temporary file.

Step 3: the business rules

PYTHON

```python
# service.py
from collections import defaultdict

def add(expenses, amount, category, note=""):
    return [*expenses, Expense(amount, category, note)]   # new list

def by_category(expenses):
    totals = defaultdict(float)
    for e in expenses:
        totals[e.category] += e.amount
    return dict(sorted(totals.items(), key=lambda kv: kv[1], reverse=True))

def summary(expenses):
    lines = [f"{cat}: {amt:.2f}" for cat, amt in by_category(expenses).items()]
    total = sum(e.amount for e in expenses)
    return "\n".join([*lines, f"TOTAL: {total:.2f}"])

items = add(add(add([], 4.5, "food"), 30, "travel"), 12.25, "food")
print(summary(items))
```

OUTPUT

```
travel: 30.00
food: 16.75
TOTAL: 46.75
```

KEY IDEA

These are **pure functions**: they depend only on their arguments and change nothing outside themselves.

DETAILS

- **add** returns a new list and leaves the old one alone.
- **defaultdict(float)** starts every category at zero.
- Sorting by the amount puts the largest category first.

Step 4: the command parser

PYTHON

```python
# cli.py (part 1)
import argparse

def build_parser():
    parser = argparse.ArgumentParser(prog="expenses")
    sub = parser.add_subparsers(dest="command", required=True)
    add_cmd = sub.add_parser("add", help="record an expense")
    add_cmd.add_argument("amount", type=float)
    add_cmd.add_argument("category")
    add_cmd.add_argument("note", nargs="?", default="")
    sub.add_parser("summary", help="show totals per category")
    return parser

args = build_parser().parse_args(["add", "4.5", "food", "coffee"])
print(args.command, args.amount, args.category, args.note)
print(build_parser().parse_args(["summary"]))
```

OUTPUT

```
add 4.5 food coffee
Namespace(command='summary')
```

VOCABULARY

argparse: the standard module that reads the words typed after the program name, converts them and writes help messages.

READING THE PARSER

- Two subcommands: **add** and **summary**.
- **type=float** converts the text **4.5** into a number.
- **nargs='?'** makes the note optional.

TIP

Passing a list to **parse_args** simulates the terminal, which makes the parser easy to try out and to test.

Step 5: wiring everything together

PYTHON

```python
# cli.py (part 2)
def main(argv=None):
    args = build_parser().parse_args(argv)
    db = Path("expenses.json")
    expenses = load(db)
    try:
        if args.command == "add":
            save(db, add(expenses, args.amount, args.category, args.note))
            print("saved")
        else:
            print(summary(expenses))
    except ValueError as error:
        print("error:", error)

main(["add", "4.5", "food", "coffee"])
main(["add", "30", "travel"])
main(["add", "0", "food"])
main(["summary"])
```

OUTPUT

```
saved
saved
error: amount must be positive
travel: 30.00
food: 4.50
TOTAL: 34.50
```

KEY IDEA

main contains no rules of its own. It only connects the parser, the storage and the service functions.

TIP

End the real file with the **__main__** guard from Module 7 and a call to **main()**.

Step 6: tests for the whole project

PYTHON

```python
# test_expenses.py
import pytest

def test_add_returns_a_new_list():
    before = []
    after = add(before, 4.5, "food")
    assert before == [] and len(after) == 1

def test_totals_are_grouped_and_sorted():
    items = add(add(add([], 4.5, "food"), 30, "travel"), 12.25, "food")
    assert by_category(items) == {"travel": 30, "food": 16.75}

def test_zero_amount_is_rejected():
    with pytest.raises(ValueError):
        Expense(0, "food")

def test_save_then_load_round_trip(tmp_path):
    db = tmp_path / "expenses.json"
    items = add([], 9.99, "books", "Python guide")
    save(db, items)
    assert load(db) == items
```

TERMINAL: pytest

```
4 passed in 0.02s
```

KEY IDEA

Four short tests cover the rules, the validation and the storage. They run in a fraction of a second.

TIP

tmp_path keeps the tests away from your real data file.

TRY IT YOURSELF

Add a **list** command that prints every expense, and write its test **first**.

Where to go next

Python is the foundation for every one of these fields. Pick the one that excites you and build something small in it this week.

1 Web development

Flask and FastAPI for services and APIs. Django for complete web applications.

2 Data analysis

NumPy for arrays, pandas for tables, Matplotlib and Plotly for charts, Jupyter for notebooks.

3 Machine learning

PyTorch, TensorFlow and XGBoost, built on the data tools in the card before this one.

4 Automation

Requests and HTTPX for web calls, Beautiful Soup for scraping, Playwright for browsers.

5 Desktop and games

Tkinter ships with Python. PySide builds rich desktop apps and Pygame builds games.

6 Cloud and operations

Boto3 for cloud services, Ansible for servers and Docker for packaging your applications.

TIP

Choose a project that you personally want to exist. Motivation carries you through the difficult middle part.

Practice challenges for every level

Level	Challenge	Skills you will practise
Beginner	A number guessing game with a limited number of attempts	Loops, conditions, input, the **random** module
Beginner	A word frequency counter for any text file	Files, dictionaries, sorting, string methods
Beginner	A temperature and currency converter with a menu	Functions, validation, f strings
Intermediate	A to do list that is saved as JSON	Classes, JSON, argparse, exceptions
Intermediate	A log file analyser that reports the busiest hours	Generators, regular expressions, **Counter**
Intermediate	A bank account simulator with a full test suite	Properties, custom exceptions, pytest
Advanced	A small library of decorators: retry, timing and caching	Closures, **functools.wraps**, type hints
Advanced	A website checker that tests one hundred addresses at once	asyncio or thread pools, timeouts
Advanced	A plugin system that discovers and registers new commands	Protocols, **__init_subclass__**, packages

The Zen of Python

Type **import this** in the shell to read the nineteen guiding principles written by Tim Peters. Six of them sum up the spirit of this tutorial.

PRINCIPLE 1

Beautiful is better than ugly.

PRINCIPLE 2

Explicit is better than implicit.

PRINCIPLE 3

Simple is better than complex.

PRINCIPLE 4

Readability counts.

PRINCIPLE 5

Errors should never pass silently.

PRINCIPLE 6

Now is better than never.

What you can do now

1. Write clear programs with variables, conditions, loops, functions and the four core containers.
2. Organise code into modules and packages and manage dependencies in virtual environments.
3. Read and write files, CSV and JSON, and handle every error with care.
4. Design classes with properties, inheritance, special methods and dataclasses.
5. Use generators, decorators, context managers and type hints like an experienced developer.
6. Explain what the interpreter does: objects, attribute lookup, memory, bytecode and the GIL.
7. Run work concurrently with threads, processes and asyncio.
8. Test, debug, profile and ship a complete command line application.

Coming next: your own project. Choose one idea today and write its first ten lines.

END OF THE TUTORIAL

Keep building

You began with a single print call and finished with coroutines, metaclasses and a tested application.

17 MODULES COMPLETE

EVERY EXAMPLE EXECUTED

Thank you for learning Python with this tutorial.

YOUR NEXT FOUR MOVES

1. Rebuild the capstone project from memory, without looking at the slides.
2. Solve three practice challenges at each level.
3. Read the source code of one library that you use.
4. Teach one topic to somebody else. Teaching reveals every gap.